MAR ATHANASIUS COLLEGE OF ENGINEERING
KOTHAMANGALAM

(Government Aided Autonomous Institution)



Department of Computer Applications

Mini Project Report

MEDICAL SPECIALTY CLASSIFICATION USING TF-IDF AND ENSEMBLE MACHINE LEARNING

done by

JIPHIN GEORGE

Reg No: MAC25MCA-2033

Under the guidance of

Prof. Biju Skaria

2025–2027

MAR ATHANASIUS COLLEGE OF ENGINEERING
KOTHAMANGALAM

(Government Aided Autonomous Institution)

CERTIFICATE

MEDICAL SPECIALTY CLASSIFICATION USING TF-IDF AND ENSEMBLE MACHINE LEARNING

Certified that this is the bonafide record of the Mini Project work carried out by

JIPHIN GEORGE

Reg No: MAC25MCA-2033

during the third semester, in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Prof. Biju Skaria
Faculty Guide

Dr. Sonia Abraham
Head of the Department

Dr. Manu John
Project Coordinator

Internal Examiner

Acknowledgement

I express my deepest praise, gratitude, and devotion to God Almighty for His boundless grace, wisdom, and blessings that guided me throughout the conceptualization, execution, and successful completion of this mini project.

I place on record my sincere thanks and respectful gratitude to Dr. Sonia Abraham, Professor and Head of the Department of Computer Applications, for her constant support, valuable encouragement, academic counsel, and for ensuring access to all necessary computing and software resources.

I am immensely indebted to my esteemed faculty guide, Prof. Biju Skaria, Associate Professor in the Department of Computer Applications, whose insightful guidance, rigorous technical feedback, continuous encouragement, and detailed supervision were vital to every stage of this research. His constructive suggestions during data preparation, feature leakage auditing, and model benchmarking contributed immeasurably to the success of this study.

I also extend my heartfelt thanks to the Project Coordinators, Dr. Manu John and Prof. Minu Maria Joseph, and all the faculty and non-teaching members of the Department of Computer Applications for their timely support, co-operation, and assistance throughout the semester.

Finally, I convey my deepest appreciation to my beloved parents, family members, and friends for their enduring patience, unconditional love, moral support, and motivation, which served as a constant pillar of strength throughout this project.

Jiphin George

Abstract

The rapid digitisation of modern healthcare has led to an exponential influx of unstructured clinical narratives within Electronic Health Record (EHR) systems, including operative reports, consultation summaries, and discharge notes. Manually triaging these clinical texts into medical specialties is labor-intensive, error-prone, and bottlenecked by inter-annotator variability. This project develops and deploys MedSpecialty AI, an automated clinical decision-support platform for medical specialty classification using sublinear Term Frequency–Inverse Document Frequency (TF-IDF) feature engineering paired with an Ensemble Machine Learning architecture.

The system utilizes the MTSamples clinical transcription corpus (4,999 raw records across 40 categories). Systematic exploratory data analysis revealed procedural ambiguity in administrative categories (e.g., General Surgery, Consult notes). A curated benchmark of 1,663 records across eight mutually exclusive hospital departments was established: Cardiovascular / Pulmonary, Orthopedic, Gastroenterology, Neurology, Urology, Obstetrics / Gynecology, ENT – Otolaryngology, and Ophthalmology. The NLP pipeline performs text cleaning, stopword removal, morphological lemmatization, and sublinear TF-IDF vectorization across an 8,000-dimensional unigram/bigram vocabulary (L_2 normalized). Four estimators were evaluated under stratified holdout validation (80% train / 20% test): Linear SVM (with Platt calibration, 88.59% accuracy), Logistic Regression, Random Forest, and Multinomial Naive Bayes. A Soft Voting Ensemble combining all four models [2:2:1:1] achieved 87.99% accuracy and a Macro F1-score of 0.8898 with calibrated posterior probabilities.

To guarantee patient safety in open-world hospital workflows, the platform incorporates an Open-Set Confidence Rejection Gate ($\tau = 0.48$). Dictations with peak ensemble confidence below 48% are routed to a 9th operational class, *"Other / Requires Review"*, intercepting unsupported clinical specialties (e.g., Dermatology, Psychiatry) and non-medical text without false specialty force-fitting. The platform is operationalized as a responsive web service featuring in-memory PyMuPDF PDF extraction, 7-phase operational dataflow tracking, and sub-10 ms CPU inference latency.

List of Tables

1 Summary of Paper 1 7

2 Summary of Paper 2 8

3 Summary of Paper 3 10

4 Overall Summary of Literature and Relation to Proposed System 11

5 Characteristics of the Kaggle Medical Transcriptions Dataset 16

6 Dataset Attribute Specifications and Missing Value Audit 17

7 Distribution of Top 10 Majority and Bottom 10 Minority Categories in Raw Dataset 18

8 Record Distribution Across the 8 Curated Clinical Specialties 20

9 Step-by-Step Clinical Text Preprocessing Examples 24

10 Project Timeline and Milestone Schedule 35

11 Software Environment and Tool Specifications 37

12 Hardware Specifications and Environment Requirements 38

13 Stratified Training and Testing Split Distribution across 8 Clinical Specialties . . . 43

14 Empirical Performance Comparison of Base Models and Voting Ensembles (8 Specialties) 47

15 Granular Classification Report and Metrics Derived from the Confusion Matrix . . 51

16 RESTful API Endpoint Specifications for the Deployed Service 56

17 In-Memory Clinical Document Processing Algorithm 57

18 Computational Latency Profiling of the Deployed Web Pipeline 61

19 Chronological Development Milestones of the MedSpecialty AI Project 69

20 Audit Trail of Significant Git Commits Documenting Project Evolution 70

21 Minimum and Standard Software Specifications 78

22 Minimum and Recommended Hardware Specifications 79

List of Figures

1 Missing Values by Dataset Attribute 20

2 Top 10 Medical Specialties by Number of Transcriptions 21

3 Distribution of All 40 Categories in Raw MTSamples Corpus 21

4 Distribution of Clinical Transcription Word Length 22

5 Transcription Length Distribution Across Top Medical Specialties 22

6 Common Lexical Terms in Medical Transcriptions 23

7 Activity Diagram of Support Vector Machine Classifier 27

8 Activity Diagram of Random Forest Classifier 28

9 Activity Diagram of Multinomial Logistic Regression Classifier 29

10 Activity Diagram of Multinomial Naive Bayes Classifier 30

11 Activity Diagram of Sublinear TF-IDF Feature Vectorizer 31

12 Activity Diagram of Calibrated Soft Voting Ensemble Classifier 32

13 Project Pipeline Diagram of Medical Specialty Classification System 33

14 System Design Training, Validation, and Partitioning Workflow 43

15 Model Performance Comparison across Accuracy, Macro F1, and Macro Recall against the 75% Target Threshold 48

16 Confusion Matrix of the Soft Voting Ensemble across 8 Clinical Specialties on the Held-Out Test Set 50

17 Screen 1: Interactive Clinical Specialty Prediction and Triage Console 63

18 Screen 2: Operational Dataflow and 7-Stage Architectural Visualizer 64

19 Screen 3: Model Performance Analytics and Confusion Matrix Dashboard 65

20 Screen 4: Vocabulary Inspector and Diagnostic Term Space Explorer 66

Contents

Acknowledgement i

Abstract ii

List of Tables iii

List of Figures iv

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Need for the System 2

1.4 Objectives 2

1.5 Scope 3

1.6 Project Overview 4

1.7 Expected Outcome 5

2 Supporting Literature 6

2.1 Literature Review 6

2.2 Literature Review Summary 10

2.3 Findings and Proposals 12

2.3.1 Findings 12

2.3.2 Proposal 13

3 System Analysis 15

3.1 Analysis of the dataset 15

3.1.1 About the dataset 15

3.1.2 Explore the dataset 17

3.1.3 Insights and conclusions of Exploratory Data Analysis . . . 18
3.2 Data Visualization . . . 20
3.3 Data Preprocessing . . . 23
3.3.1 Data Cleaning . . . 23
3.3.2 Analysis of Feature Variables . . . 24
3.3.3 Analysis of Class Variables . . . 25
3.4 Analysis of Architecture and Algorithms . . . 26
3.4.1 Detailed Study Report of the Proposed Algorithms . . . 26
3.5 Project pipeline . . . 33
3.6 Project Timeline . . . 35
3.7 Feasibility Analysis . . . 36
3.7.1 Technical Feasibility . . . 36
3.7.2 Economic Feasibility . . . 36
3.7.3 Operational Feasibility . . . 36
3.7.4 Schedule Feasibility . . . 36
3.8 System Environment . . . 37
3.8.1 Software Environment . . . 37
3.8.2 Hardware Environment . . . 38

4 System Design **39**

4.1 Model(s) Building . . . 39
4.1.1 Implementation Code . . . 39
4.1.2 Model Planning (Training/Testing Split of the Dataset) . . . 42
4.1.3 Training . . . 43
4.1.4 Testing . . . 45

5 Results and Discussion **47**

5.1 Comparison of Models . . . 47

5.2 Evaluation Metrics 48
5.3 Confusion Matrix 49
5.4 Relevant Metrics Calculated from Confusion Matrix 50
5.5 Prediction Statistics of Unseen Data 52
5.6 Discuss Your Findings and Conclusions 53

6 Model Deployment **55**
6.1 Model Implementation 55
6.1.1 Web Service Architecture and Framework Selection 55
6.1.2 In-Memory Clinical PDF and Document Text Parsing 56
6.1.3 Pipeline Serialization and Model Deserialization 57
6.1.4 RESTful API Specification 58
6.1.5 Open-Set Rejection and Clinical Safety Gating 60
6.1.6 Computational Latency and Resource Profiling 60
6.2 User Interface (UI) Design 61
6.2.1 Design Philosophy and Clinical Usability 61
6.2.2 Dashboard Layout and Primary Interaction Modules 62
6.2.3 Screen 1: Clinical Prediction Dashboard 62
6.2.4 Screen 2: Operational Dataflow Visualizer (How It Works) 64
6.2.5 Screen 3: Model Performance Analytics Dashboard 65
6.2.6 Screen 4: Feature Space and Vocabulary Inspector 65

7 Git History **67**
7.1 Version Control Strategy and Repository Architecture 67
7.1.1 Branching Strategy and Workflow 67
7.1.2 Commit Conventions and Semantic Versioning 67
7.2 Chronological Project Development Milestones 68
7.3 Detailed Git Commit History and Audit Trail 69

7.4 Repository Metrics and Code Quality Assurance 71

8 Conclusions **72**

8.1 Summary of Technical Achievements 72

8.2 Significance and Impact on Clinical Workflow 73

9 Future Scope **75**

9.1 Multi-Modal Clinical Diagnostic Ingestion 75

9.2 Edge Computing and On-Device Mobile Healthcare Deployment 75

9.3 HL7 FHIR Interoperability and Hospital EHR Integration 76

9.4 Active Learning and Human-in-the-Loop Continuous Retraining 76

9.5 Multi-Lingual Clinical Translation and Voice Dictation 77

10 Appendix **78**

10.1 Minimum Software Requirements 78

10.2 Minimum Hardware Requirements 79

11 References **80**

1. Introduction

1.1 Background

In contemporary healthcare institutions, the widespread adoption of Electronic Health Record (EHR) and Electronic Medical Record (EMR) systems has resulted in the continuous generation of massive volumes of clinical narratives. Healthcare practitioners generate extensive unstructured text daily, including admission notes, preoperative consultations, surgical operative summaries, progress notes, and diagnostic evaluations. Accurately categorizing these unstructured clinical notes into specific medical specialties (such as Cardiology, Neurology, Orthopedics, Gastroenterology, Urology, and Ophthalmology) is an indispensable prerequisite for hospital workflow automation, patient file routing, diagnostic database indexing, and clinical decision support.

Traditionally, document indexing relies on manual review by medical coders and health information management personnel. With escalating patient volumes in modern medical centers, manual categorization has become a severe administrative bottleneck, introducing substantial latency, high labor costs, and inter-annotator classification discrepancies. Natural Language Processing (NLP) paired with Machine Learning (ML) provides an automated computational paradigm capable of rapidly extracting semantic signals from clinical narratives, standardizing specialty categorization, and routing medical documentation in real time.

1.2 Problem Statement

Manual categorization of medical transcription narratives into appropriate clinical specialties is labor-intensive, time-consuming, and prone to subjective diagnostic interpretation. Unstructured clinical notes present formidable linguistic complexities, including dense domain-specific medical terminology, non-standard abbreviations, variable document lengths, and anatomical lexical overlap across adjacent clinical sub-disciplines.

Furthermore, authentic medical transcription datasets suffer from severe class imbalance and noisy labeling. Datasets such as the Kaggle Medical Transcriptions (MTSamples) corpus initially group notes into broad procedural or administrative categories (such as General Surgery, SOAP Notes, and Consultative Summaries), which overlap heavily with organ-specific departments. A critical limitation of conventional machine learning systems is their closed-set as-

sumption: they assume that every incoming clinical record belongs to one of the pre-defined target classes. In real-world hospital environments, out-of-scope clinical narratives (such as Dermatology, Psychiatry, or Pediatrics) or non-medical text are routinely encountered. Closed-set classifiers mistakenly force-fit these out-of-domain cases into an arbitrary specialty, compromising clinical integrity. Additionally, deep learning transformer models (such as BERT or BioBERT) incur excessive GPU computational overhead, rendering them impractical for lightweight, instantaneous web deployment.

1.3 Need for the System

Automating medical specialty classification through efficient statistical natural language processing and ensemble machine learning addresses critical clinical needs:

- *Streamlined Clinical Triage*: Eliminates manual document sorting backlogs, enabling automated indexing and immediate department routing upon clinical dictation.
- *Rapid Emergency Retrieval*: Empowers attending physicians to quickly access historical specialty-specific operative notes during acute patient emergencies.
- *Open-Set Patient Safety*: Implements a confidence-gated rejection mechanism that identifies out-of-scope or ambiguous medical documents, safely routing them to manual clinical review rather than producing hazardous misclassifications.
- *Explainable NLP Representation*: Provides algorithmic transparency through Term Frequency–Inverse Document Frequency (TF-IDF) feature attribution, allowing clinicians to inspect the exact keywords driving model decisions.
- *Computationally Efficient Web Deployment*: Delivers instantaneous inference on standard CPU architectures through lightweight classical estimators, hosted via an accessible web platform with direct PDF report ingestion.

1.4 Objectives

The primary objective of this project is to design, develop, and deploy a robust, end-to-end automated Medical Specialty Classification system (**MedSpecialty AI Platform**) using sublinear TF-IDF vectorization and an Ensemble Machine Learning architecture with open-set rejection capabilities. The specific objectives are:

1. To conduct comprehensive Exploratory Data Analysis (EDA) on the raw MTSamples medical transcriptions corpus (4,999 records across 40 categories) and identify noisy, overlapping, and administrative document classes.

2. To curate a clinically sound, mutually exclusive 8-specialty benchmark dataset comprising 1,663 records across core hospital departments.

3. To build a modular clinical NLP preprocessing pipeline incorporating lowercasing, medical punctuation handling, tokenization, stopword removal, and morphological lemmatization.

4. To extract discriminative unigram and bigram features using an 8,000-dimensional sublinear TF-IDF vectorizer with L_2 Euclidean normalization.

5. To train and evaluate candidate supervised classifiers with distinct inductive biases: Linear Support Vector Machine (Linear SVM with Platt probability calibration), Logistic Regression, Random Forest, and Multinomial Naive Bayes.

6. To construct a Soft Voting Ensemble Classifier utilizing calibrated weights [2:2:1:1] to synthesize probabilistic consensus across candidate models.

7. To formulate an Open-Set Confidence Rejection mechanism with a calibrated safety threshold ($\tau = 0.48$) establishing a 9th operational class (*"Other / Requires Review"*) for out-of-scope clinical records.

8. To develop and deploy an interactive, production-grade web application using Flask, supporting drag-and-drop PDF text extraction via PyMuPDF, real-time transcription editing, and 7-phase dataflow visualization.

1.5 Scope

The scope of this project encompasses:

- *Dataset Scope*: Evaluates 4,999 raw clinical transcription records from the MTSamples corpus, filtering out administrative notes (SOAP records, consultations, summaries) and overlapping categories (General Surgery) to establish a benchmark of 1,663 records across eight core hospital departments:

1. Cardiovascular / Pulmonary
2. Orthopedic
3. Gastroenterology
4. Neurology
5. Urology
6. Obstetrics / Gynecology
7. ENT – Otolaryngology
8. Ophthalmology

- *Feature Engineering Scope*: Focuses on unigram and bigram TF-IDF representations bounded at 8,000 features, applying logarithmic term frequency dampening $(1+\log(\text{TF}))$ to prevent high-frequency term domination.
- *Algorithmic Scope*: Investigates four classical machine learning models (Linear SVM, Logistic Regression, Random Forest, and Multinomial Naive Bayes), a Soft Voting Ensemble, and a threshold-based open-set rejection gate.
- *Deployment Scope*: Delivers a fully responsive web application (**MedSpecialty AI Platform**) built with Flask and Vanilla CSS/JavaScript, featuring in-memory PDF parsing (PyMuPDF) and full pipeline explainability without requiring external GPU infrastructure.

1.6 Project Overview

The project follows a structured engineering workflow:

1. *Data Ingestion & Exploratory Profiling*: Loading raw MTSamples CSV data, analyzing missing attributes, document length distributions, and category imbalances.
2. *Dataset Curation & Cleaning*: Pruning null records, resolving category ambiguities, and extracting enriched clinical text (combining consultation abstracts and transcription narratives).
3. *NLP Preprocessing*: Standardizing text via regex cleaning, lowercasing, punctuation stripping, tokenization, stopword removal, and WordNet morphological lemmatization.

4. *Feature Extraction*: Vectorizing preprocessed narratives into an 8,000-dimensional sparse feature space using sublinear TF-IDF scaling.

5. *Model Development & Calibration*: Training candidate estimators on stratified partitions (80% train / 20% test) and applying Platt scaling via `CalibratedClassifierCV` to Linear SVM.

6. *Ensemble Integration*: Aggregating model posteriors using weighted soft voting $(2 \times \text{SVM} + 2 \times \text{LR} + 1 \times \text{RF} + 1 \times \text{MNB})/6$.

7. *Open-Set Confidence Gating*: Applying decision threshold $\tau = 0.48$ to classify cases into confirmed specialties or route them to *"Other / Requires Review"*.

8. *Web Platform Deployment*: Serializing artifacts with Joblib and serving predictions via the MedSpecialty AI Platform with PDF ingestion.

1.7 Expected Outcome

The project delivers the following primary outcomes:

- A curated, high-quality benchmark dataset of 1,663 medical transcriptions across eight clinical specialties.
- An empirical comparative analysis confirming that Linear SVM achieves the highest individual accuracy (88.59%, Macro F1: 0.8971) and that the Soft Voting Ensemble achieves superior probability calibration (87.99%, Macro F1: 0.8898).
- A verified open-set rejection mechanism that accurately routes unsupported clinical records (e.g., Dermatology, Psychiatry) to the *"Other"* category.
- A fully functional, production-ready web application providing real-time PDF extraction and clinical NLP explainability.
- Comprehensive academic documentation adhering to MCA departmental standards.

2. Supporting Literature

2.1 Literature Review

A systematic review of contemporary literature was conducted to evaluate existing methodological frameworks, feature engineering representations, classification algorithms, class imbalance mitigation techniques, and dataset challenges in clinical natural language processing. The review focuses on peer-reviewed research analyzing medical narratives, electronic health records, and document categorization systems.

Paper 1 – Omar AM, Elhoseny M, Hassanien AM. Clinical text classification with word representation features and machine learning algorithms. International Journal of Online and Biomedical Engineering (iJOE). 2023.

The paper titled "Clinical Text Classification with Word Representation Features and Machine Learning Algorithms" [1] investigates the impact of diverse text representation techniques on Electronic Medical Record (EMR) narratives. The authors introduced a structured four-phase classification framework consisting of clinical text preprocessing, word vectorization, dimensionality reduction, and supervised classification.

To model clinical narratives, the study conducted a comparative assessment of classical Bag-of-Words (BoW) and Term Frequency–Inverse Document Frequency (TF-IDF) representations against dense distributed semantic embeddings (Word2Vec) across multiple supervised estimators, including Logistic Regression, Support Vector Machine (SVM), Naive Bayes, and k-Nearest Neighbors (k-NN). The empirical evaluations indicated that pairing continuous Word2Vec embeddings with a k-NN classifier yielded the highest classification accuracy of 92%, demonstrating that continuous representations capture semantic affinities between medical terms more effectively than unigram Bag-of-Words.

However, the authors noted that distance-based k-NN introduces significant computational latency during test-time inference, scaling poorly to large-scale hospital databases. Furthermore, the investigation evaluated individual classifiers in isolation without exploring multi-model ensemble consensus or real-time web deployment frameworks. Crucially, Principal Component Analysis (PCA) was evaluated exclusively as an offline dimensionality reduction step in Paper 1 and is not required in our proposed architecture, as sublinear TF-IDF with feature pruning

maintains optimal sparsity without dense projection.

TITLE	Clinical Text Classification with Word Representation Features and Machine Learning Algorithms [1]
AREA OF WORK	Clinical Natural Language Processing / EMR Text Vectorization
DATASET	Electronic Medical Record (EMR) Clinical Transcriptions Dataset
METHODOLOGY / STRATEGY	4-Phase Framework: (1) Preprocessing, (2) Vectorization (BoW, TF-IDF, Word2Vec), (3) Feature Reduction (PCA), (4) Supervised Classification
ALGORITHM	Logistic Regression, Support Vector Machine (SVM), Naive Bayes, k-NN, PCA
RESULT / PRECISION	Word2Vec + k-NN achieved the highest classification accuracy of 92%
ADVANTAGES	Proves dense semantic representations capture clinical context better than raw unigram BoW
LIMITATIONS	Distance-based k-NN exhibits high inference latency; lacks ensemble voting, open-set handling, and real-time web deployment
RELEVANCE TO MY PROJECT	Validates the standard 4-phase clinical text preprocessing sequence and benchmark machine learning estimators
FUTURE PROPOSAL	Develop lightweight ensemble classifiers suitable for instantaneous CPU web inference with calibrated confidence

Table 1: Summary of Paper 1

Paper 2 – Zhang H, Zhu D, Tan H, Shafiq M, Gu Z. Medical specialty classification based on semiadversarial data augmentation. Hindawi Computational Intelligence and Neuroscience. 2023.

The research article titled "Medical Specialty Classification Based on Semiadversarial Data Augmentation" [2] investigated extreme category imbalance and data sparsity in clinical specialty categorization. The authors introduced Semiadversarial Data Augmentation (SemiADA), a framework generating synthetic clinical text samples along adversarial perturbation trajectories to populate sparse decision regions in high-dimensional feature spaces.

Additionally, a Probabilistic Information (PI) layer was incorporated following the Softmax output to recalculate prediction confidence by weighting domain-specific clinical nouns. The authors evaluated a filtered subset of the Kaggle Medical Specialty Classification dataset re-

stricted to 18 classes, achieving an average performance improvement of 14.9% in accuracy and F1-score over baseline benchmarks. Their findings confirmed that domain-specific clinical nouns carry the primary discriminative signal for medical department routing.

Despite these gains, generating synthetic samples along adversarial attack paths incurs extreme computational overhead. Furthermore, fine-tuning deep transformer architectures (such as BERT or BioBERT) requires dedicated GPU clusters and introduces substantial inference latency, creating a major barrier for real-time clinical web applications operating in resource-constrained hospital environments. The study also operated strictly under a closed-set assumption, omitting mechanisms to intercept out-of-domain records.

TITLE	Medical Specialty Classification Based on Semiadversarial Data Augmentation [2]
AREA OF WORK	Clinical Text Data Augmentation / Severe Class Imbalance Mitigation
DATASET	Kaggle Medical Specialty Classification Dataset (Subset restricted to 18 classes)
METHODOLOGY / STRATEGY	Semiadversarial sample generation along attack trajectories + Probabilistic Information (PI) noun-weighted Softmax recalculation
ALGORITHM	SemiADA, BERT (Transformer), Softmax with PI Noun Layer
RESULT / PRECISION	Achieved an average 14.9% boost in Accuracy and Macro F1-score over baseline benchmarks
ADVANTAGES	Populates sparse minority decision boundaries using geometry-aware synthetic clinical samples
LIMITATIONS	Extreme training latency; fine-tuning BERT demands dedicated GPU accelerators; closed-set classification only
RELEVANCE TO MY PROJECT	Highlights class imbalance and noun significance in MTSamples data; motivates high-dimensional TF-IDF with ensemble ML
FUTURE PROPOSAL	Combine sublinear TF-IDF n-gram weighting with probability-calibrated ensemble learning and open-set rejection on standard CPUs

Table 2: Summary of Paper 2

Paper 3 – Blanchard AE, Gao S, Yoon HJ, Tourassi G. A keyword-enhanced approach to handle class imbalance in clinical text classification. IEEE Journal of Biomedical and Health Informatics. 2022.

Blanchard et al., writing in the *IEEE Journal of Biomedical and Health Informatics* [3], addressed the challenge of deep neural networks overfitting on rare clinical categories when trained on lengthy, complex pathology narratives. The authors introduced a data-centric regularisation strategy by injecting short class-associated keyword sequences into mini-batches during neural network training, formulating a dual loss function:

$$L = L_{\text{docs}} + \alpha L_{\text{key}} \tag{1}$$

where L_{docs} evaluates cross-entropy loss over full documents, L_{key} penalizes misclassifications on isolated diagnostic keyword sequences, and α balances the objective.

Evaluating on the National Cancer Institute (NCI) Surveillance, Epidemiology, and End Results (SEER) cancer pathology registry dataset, the study benchmarked Convolutional Neural Networks (CNNs), keyword-constrained loss functions, and conventional resampling techniques (oversampling and undersampling). The findings demonstrated that keyword constraints dramatically improved Macro F1-score and minority class recall without degrading majority class performance. Crucially, the authors emphasized that overall micro-accuracy is an inherently deceptive evaluation metric in clinical NLP, as high global accuracy frequently masks near-zero recall on rare clinical conditions.

However, their approach relies on manual curation of comprehensive keyword lexicons by clinical domain experts for every target category, limiting rapid scalability to new medical specialties. This limitation directly motivates our system's automated statistical TF-IDF approach, which extracts informative unigram and bigram vocabulary automatically without requiring hand-crafted dictionaries.

TITLE	A Keyword-Enhanced Approach to Handle Class Imbalance in Clinical Text Classification [3]
AREA OF WORK	Imbalanced Clinical Narrative Categorization / Pathology NLP
DATASET	NCI SEER Surveillance Cancer Pathology Registry Reports Dataset
METHODOLOGY / STRATEGY	Mini-batch keyword sequence injection to constrain the neural loss function and prevent rare-class forgetting
ALGORITHM	Convolutional Neural Networks (CNNs), Keyword-Constrained Loss Optimization
RESULT / PRECISION	Significantly boosted Macro F1-score and minority recall across imbalanced pathology classes
ADVANTAGES	Demonstrates that keyword and n-gram terms carry decisive clinical signals; confirms Macro F1 as the primary clinical metric
LIMITATIONS	Requires exhaustive manual curation of keyword dictionaries by medical specialists; computationally heavy neural training
RELEVANCE TO MY PROJECT	Proves that n-gram keyword weighting preserves rare class discrimination; validates Macro F1 as our primary evaluation metric
FUTURE PROPOSAL	Implement automated statistical TF-IDF unigram and bigram extraction with sublinear scaling, eliminating manual dictionary construction

Table 3: Summary of Paper 3

2.2 Literature Review Summary

Table 4 presents a comparative synthesis of the reviewed literature alongside our proposed MedSpecialty AI Platform.

Paper	**Area**	**Dataset**	**Algorithm**	**Advantages**	**Relation to Proposed Work**
Omar et al. [1]	Clinical Text Vectorization	EMR Text	LR, SVM, NB, k-NN	High accuracy (92%) with Word2Vec	Establishes the 4-phase text preprocessing pipeline
Zhang et al. [2]	Data Augmentation	MTSamples (18 classes)	SemiADA, BERT	+14.9% gain in Accuracy and F1	Identifies MTSamples class imbalance and clinical noun signal
Blanchard et al. [3]	Class Imbalance Regularization	NCI SEER Pathology	CNN + Keyword Loss	High Macro F1 on minority classes	Establishes Macro F1 as primary metric; validates n-gram term weighting
Proposed Work	**Specialty Triage & Open-Set Routing**	**MTSamples (Curated 8 Core + Other)**	**Sublinear TF-IDF (8k) + Soft Voting Ensemble [2:2:1:1]**	**Fast CPU inference, open-set safety gate ($\tau = 0.48$), interactive web UI**	**Complete, verified clinical decision support platform (MedSpecialty AI)**

Table 4: Overall Summary of Literature and Relation to Proposed System

Evolution of Research

Clinical natural language processing has evolved from elementary Bag-of-Words term frequency counters toward distributed word embeddings (Word2Vec), deep convolutional architectures, complex transformer models (BERT, BioBERT), and adversarial data augmentation (SemiADA). While deep neural networks deliver high semantic abstraction, their steep computational overhead, large parameter footprints, and GPU hardware prerequisites prevent real-time deployment in cost-conscious clinical web environments. Furthermore, deep neural networks often suffer from overconfidence on out-of-domain text.

Identified Research Gaps

The literature review reveals four fundamental research gaps:

1. *Excessive Computational Overhead*: Existing deep learning systems rely on GPU hardware accelerators and introduce substantial inference latency, making real-time interactive dictation triage challenging.

2. *Closed-Set Vulnerability*: Prior works operate exclusively under a closed-set assumption, failing to handle out-of-scope medical specialties or non-clinical narratives, resulting in hazardous false specialty assignments.

3. *Lack of End-to-End Clinical Deployment*: Most studies focus solely on offline experimental benchmarks without delivering an operational, accessible web application supporting direct PDF ingestion.

4. *Single Estimator Bias*: Many prior systems rely on a single classifier architecture, failing to exploit ensemble diversity to balance high-margin boundaries with calibrated probabilistic spreads.

Motivation for Proposed System

Our project directly addresses these research gaps by designing the **MedSpecialty AI Platform**. The system couples an 8,000-dimensional sublinear TF-IDF vectorizer (capturing unigram terms and bigram clinical phrases) with a calibrated Soft Voting Ensemble aggregating Linear SVM, Logistic Regression, Random Forest, and Multinomial Naive Bayes using optimal weights [2:2:1:1]. Crucially, an Open-Set Confidence Rejection Gate ($\tau = 0.48$) is established to route unsupported or ambiguous narratives to a dedicated *"Other / Requires Review"* class. The entire architecture is deployed through a responsive Flask web application featuring PyMuPDF in-memory text parsing and real-time NLP pipeline inspection on standard CPU hardware.

2.3 Findings and Proposals

2.3.1 Findings

The systematic review of contemporary literature yields several critical findings:

1. *Efficacy of Classical ML with N-Gram Term Weighting*: Classical machine learning algorithms (Support Vector Machines, Logistic Regression, Random Forest, and Naive Bayes) demonstrate competitive classification performance on clinical narratives when paired with sublinear TF-IDF feature representations, matching or exceeding heavy neural models on moderate-sized corpora while executing in milliseconds on standard CPUs.

2. *Sublinear TF Dampening Prevents Frequency Bias*: Clinical dictations feature frequent repetitions of routine procedural terms. Sublinear logarithmic scaling $(1 + \log(\text{TF}))$ successfully prevents high-frequency terms from overshadowing specialty-defining diagnostic keywords.

3. *Complementary Estimator Inductive Biases*: Different classifier families exhibit complementary strengths. Linear SVM identifies robust maximum-margin hyperplanes in high-dimensional sparse spaces; Logistic Regression provides well-calibrated odds; Random Forest captures non-linear feature interactions; and Multinomial Naive Bayes contributes probabilistic frequency baselines.

4. *Necessity of Open-Set Rejection*: Closed-set classification in clinical environments creates unacceptable diagnostic risks when processing out-of-scope conditions. An empirical confidence threshold provides an effective safeguard for triaging unrecognized narratives to human experts.

5. *Importance of Macro F1 Evaluation*: In clinical NLP with imbalanced specialty distributions, overall accuracy is deceptive. Macro F1-score—which weights each clinical department equally—serves as the primary indicator of balanced diagnostic capability.

2.3.2 Proposal

Synthesizing these findings, this project proposes the **MedSpecialty AI Platform** for automated clinical specialty classification and open-set triage:

1. *Curated Benchmark Dataset*: Filter the raw MTSamples corpus down from 40 overlapping/administrative categories to a curated benchmark of 1,663 records across eight mutually exclusive hospital specialties: Cardiovascular / Pulmonary, Orthopedic, Gastroenterology, Neurology, Urology, Obstetrics / Gynecology, ENT – Otolaryngology, and Ophthalmology.

2. *Modular NLP Preprocessing*: Implement an automated cleaning pipeline performing lowercasing, regex noise removal, tokenization, stopword elimination, and WordNet morphological lemmatization.

3. *Sublinear TF-IDF Vectorization*: Extract unigram and bigram features bounded at 8,000 dimensions with sublinear term frequency scaling, document frequency cutoffs ($\text{min_df} = 2, \text{max_df} = 0.70$), and Euclidean L_2 normalization.

4. *Calibrated Soft Voting Ensemble*: Train Linear SVM (calibrated via Platt scaling), Logistic Regression, Random Forest, and Multinomial Naive Bayes, synthesizing predictions using calibrated voting weights $[2 : 2 : 1 : 1]$.

5. *Open-Set Rejection Architecture*: Formulate a decision rule: if the peak ensemble probability $P_{\max} < 0.48$, the document is classified as *"Other"* with status *"Out of Scope / Requires Review"*, displaying nearest candidate specialties for manual triage.

6. *Interactive Web Application*: Deploy a production-grade web application featuring in-memory PDF extraction (PyMuPDF), editable clinical dictation areas, interactive TF-IDF feature tables, and multi-model consensus dashboards.

3. System Analysis

System analysis establishes the foundational functional requirements, dataset architecture, exploratory statistical properties, preprocessing transformations, algorithmic mechanics, and deployment feasibility for the automated Medical Specialty Classification system.

The empirical investigation is conducted on the Kaggle Medical Transcriptions (MTSamples) dataset [4], which provides authentic, transcribed clinical dictations across diverse healthcare disciplines. Analyzing this corpus facilitates the development of a clinically sound, computationally lightweight natural language processing pipeline capable of differentiating domain-specific medical vocabulary while remaining suitable for real-time web deployment on standard CPU hardware.

3.1 Analysis of the dataset

3.1.1 About the dataset

Dataset Overview

The Kaggle Medical Transcriptions dataset is a curated collection of authentic clinical narratives obtained from MTSamples.com, an open clinical transcription repository created to assist medical transcriptionists and healthcare informatics researchers. The dataset encapsulates a broad spectrum of real-world clinical encounters, ranging from routine consultative histories and physical examinations to highly complex intraoperative surgical summaries.

- **Total Raw Records**: 4,999 clinical transcription reports.
- **Total Attributes**: 6 attributes (1 integer identifier, 4 textual features, and 1 categorical target label).
- **Data Types**: Five text attributes: clinical narratives (`transcription`), diagnostic synopses (`description`), keywords (`keywords`), procedure titles (`sample_name`), and specialty labels (`medical_specialty`).
- **Target Attribute**: `medical_specialty`, containing 40 nominal medical department categories in the raw distribution.

Table 5 summarizes the fundamental structural characteristics of the raw dataset.

Property	Description
Dataset Name	Kaggle Medical Transcriptions (MTSamples) Dataset
Source	Kaggle (`https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions`)
Total Raw Records	4,999 clinical reports
Number of Attributes	6 fields (`Unnamed: 0`, `description`, `medical_specialty`, `sample_name`, `transcription`, `keywords`)
Primary Feature Variable	`transcription` (Unstructured clinical narrative dictation)
Auxiliary Feature Variable	`description` (Short diagnostic synopsis)
Target Class Variable	`medical_specialty` (40 raw categories; curated to 8 core specialties + 1 Open-Set class)
Advantages	Authentic clinical language; zero missing target labels (100% labeled); rich multi-specialty terminology
Challenges	Severe category imbalance; missing transcription entries (33 nulls); overlapping administrative document categories

Table 5: Characteristics of the Kaggle Medical Transcriptions Dataset

Attribute Specifications

The six dataset attributes serve distinct analytical functions:

1. **Record Identifier (`Unnamed: 0`)**: An integer sequence representing the raw tabular row index (range: 0 to 4,998).

2. **Clinical Abstract (`description`)**: A concise, one-to-two sentence clinical synopsis summarizing the patient's presenting condition, consultation findings, or surgical objective.

3. **Target Medical Specialty (`medical_specialty`)**: The categorical label designating the medical specialty or departmental division under which the transcription was categorized.

4. **Dictation Title (`sample_name`)**: A brief descriptive title denoting the specific clinical procedure, anatomical site, or encounter type (e.g., "Coronary Artery Bypass",

"Colonoscopy").

5. **Primary Clinical Narrative (`transcription`)**: The central predictor feature containing the full, free-text clinical dictation. It details the patient's medical history, physical examination, diagnostic investigations, intraoperative findings, and postoperative care plans.

6. **Extracted Keywords (`keywords`)**: Comma-delimited medical keywords and anatomical descriptors indexed from the report.

3.1.2 Explore the dataset

Statistical Data Profiling and Null Value Audit

Exploratory data profiling was conducted using Python data analysis frameworks (`pandas`, `numpy`). Systematically inspecting the schema via `shape`, `info()`, `dtypes`, and null audits confirmed the presence of 4,999 records across 6 attributes. Table 6 provides a complete audit of column types, descriptions, and missing record frequencies.

Attribute Name	Data Type	Clinical Description	Null Count	Null %
`Unnamed: 0`	Integer	Record index / sequence number	0	0.00%
`description`	Text	Short consultation summary	6	0.12%
`medical_specialty`	Categorical	Target clinical specialty	0	0.00%
`sample_name`	Text	Procedure / dictation sample title	0	0.00%
`transcription`	Text	Primary clinical narrative	33	0.66%
`keywords`	Text	Extracted clinical terms	1,068	21.36%

Table 6: Dataset Attribute Specifications and Missing Value Audit

The audit confirms that the target attribute (`medical_specialty`) is fully populated across all 4,999 instances. In the primary feature attribute (`transcription`), 33 records contain empty/null values (0.66%), which are removed during initial data pruning. In contrast, `keywords` exhibits 1,068 missing records (21.36%), justifying its exclusion as a primary feature to avoid substantial feature sparsity and imputation bias.

Class Distribution and Imbalance in Raw Dataset

Analyzing the raw target variable revealed extreme category imbalance across the 40 classes. Table 7 contrasts the top 10 majority specialties against the 10 rarest minority specialties.

Top 10 Majority Classes	Record Count	Bottom 10 Minority Classes	Record Count
Surgery	1,103	Cosmetic / Plastic Surgery	27
Consult - History and Phy.	516	Pediatric / Neonatal	22
Cardiovascular / Pulmonary	372	Sleep Medicine	20
Orthopedic	355	Bariatrics	18
Radiology	273	Speech - Language	9
General Medicine	259	Lab Medicine - Pathology	8
Gastroenterology	230	Allergy / Immunology	7
Neurology	223	Hospice - Palliative Care	6
SOAP / Chart / Progress Notes	166	Executive Evaluation	2
Obstetrics / Gynecology	160	Autopsy	2

Table 7: Distribution of Top 10 Majority and Bottom 10 Minority Categories in Raw Dataset

3.1.3 Insights and conclusions of Exploratory Data Analysis

A critical finding from data exploration is that the raw 40 classes do not represent mutually exclusive medical specialties. Instead, the raw dataset mixes anatomical medical specialties with general administrative document formats and procedural umbrellas:

1. *Administrative Document Formats*: Categories such as `Consult - History and Phy.` (516 records), `SOAP / Chart / Progress Notes` (166 records), `Discharge Summary` (108 records), `Office Notes` (50 records), and `Emergency Room Reports` (75 records) define documentation templates rather than distinct medical specialties. Reports within these classes span cardiology, neurology, gastroenterology, and orthopedics, introducing severe semantic ambiguity.

2. *Procedural Umbrella Overlap*: The largest category, `Surgery` (1,103 records), functions as an overarching procedural umbrella. An appendectomy in `Surgery` shares identical clinical terminology with `Gastroenterology`; a total knee replacement in `Surgery` overlaps with `Orthopedic`; and a coronary artery bypass graft in `Surgery` overlaps with

`Cardiovascular / Pulmonary`. Retaining `Surgery` forces classifiers to learn artificial distinctions between surgical and clinical records of the same organ system.

3. *Severe Minority Sparsity*: Several specialized classes contain fewer than 10 records (e.g., `Allergy / Immunology`: 7, `Hospice - Palliative Care`: 6, `Autopsy`: 2), making statistically sound train-test splitting impossible.

To establish a clinically coherent, academically defensible classification benchmark, a rigorous filtering methodology was applied, restricting the classification universe to the **eight core mutually exclusive clinical specialties**. These eight specialties represent distinct organ systems with established hospital departmental divisions:

1. **Cardiovascular / Pulmonary**: Heart, vascular, lung, and thoracic conditions.
2. **Orthopedic**: Musculoskeletal, joint, bone fracture, and spinal interventions.
3. **Gastroenterology**: Digestive tract, liver, colon, and endoscopy procedures.
4. **Neurology**: Brain, central nervous system, stroke, and neuromuscular diseases.
5. **Urology**: Urinary tract, kidney, bladder, and male reproductive system.
6. **Obstetrics / Gynecology**: Female reproductive system, pregnancy, and childbirth.
7. **ENT – Otolaryngology**: Ear, nose, throat, larynx, and sinonasal disorders.
8. **Ophthalmology**: Eye diseases, cataracts, retina, and vision surgery.

Following null removal and domain filtering, the finalized curated clinical dataset comprises **1,663 records**. Table 8 details the distribution of the curated benchmark.

Rank	Medical Specialty	Sample Count	Percentage (%)
1	Cardiovascular / Pulmonary	371	22.31%
2	Orthopedic	355	21.35%
3	Gastroenterology	230	13.83%
4	Neurology	223	13.41%
5	Obstetrics / Gynecology	160	9.62%
6	Urology	156	9.38%
7	ENT – Otolaryngology	96	5.77%
8	Ophthalmology	83	4.99%
	Total Curated Clinical Dataset	**1,663**	**100.00%**

Table 8: Record Distribution Across the 8 Curated Clinical Specialties

3.2 Data Visualization

Exploratory data visualization reveals structural, lexical, and distributional patterns across the clinical transcription corpus.

Missing Value Distribution

Figure 1 displays the missing record counts across attributes. While `medical_specialty` contains zero missing entries and `transcription` has only 33 nulls (0.66%), the auxiliary `keywords` field exhibits 1,068 missing records (21.36%), confirming that keywords cannot serve as a reliable primary classification attribute.

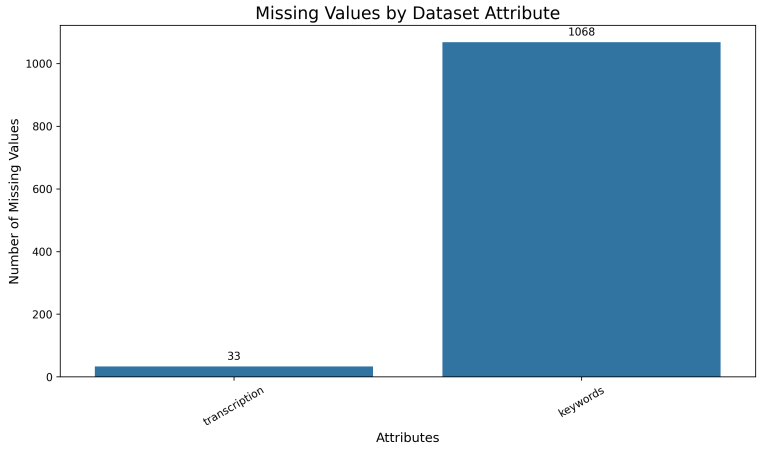


Figure 1: Missing Values by Dataset Attribute

Top 10 Medical Categories

Figure 2 illustrates transcription frequencies for the top 10 categories in the raw dataset. Surgery dominates with 1,103 records, followed by Consult - History and Phy. (516 records), Cardiovascular / Pulmonary (372 records), and Orthopedic (355 records).

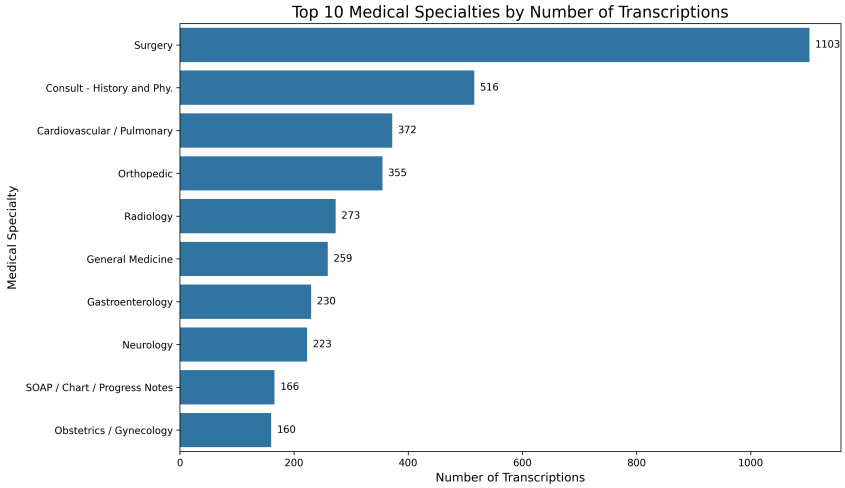


Figure 2: Top 10 Medical Specialties by Number of Transcriptions

Complete Class Distribution (40 Categories)

Figure 3 illustrates the severe class imbalance across all 40 raw categories, underscoring why filtering to 8 mutually exclusive specialties was essential for academic validity.

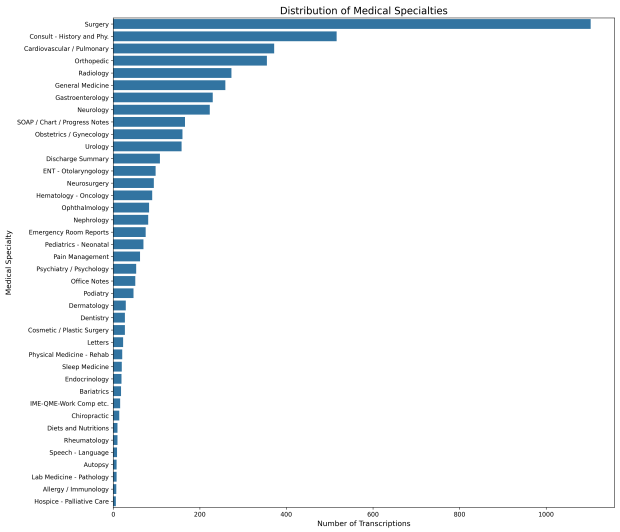


Figure 3: Distribution of All 40 Categories in Raw MTSamples Corpus

Clinical Narrative Length Distribution

Figure 4 displays the word count distribution for clinical narratives, fitted with a Kernel Density Estimate (KDE) curve. The distribution is right-skewed; most transcriptions contain between 200 and 600 words, with comprehensive operative notes extending beyond 2,500 words.

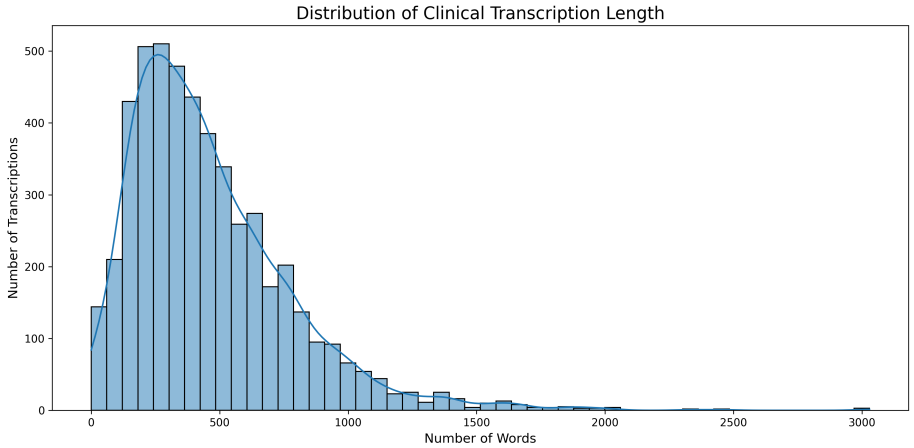


Figure 4: Distribution of Clinical Transcription Word Length

Document Length Variations Across Specialties

Figure 5 presents box plots comparing narrative word count distributions across specialties. Consultations and diagnostic notes (e.g., Radiology) feature shorter median lengths (200 words), whereas Orthopedic and Cardiovascular operative summaries are substantially longer (600 words), emphasizing the need for L_2 length normalization.

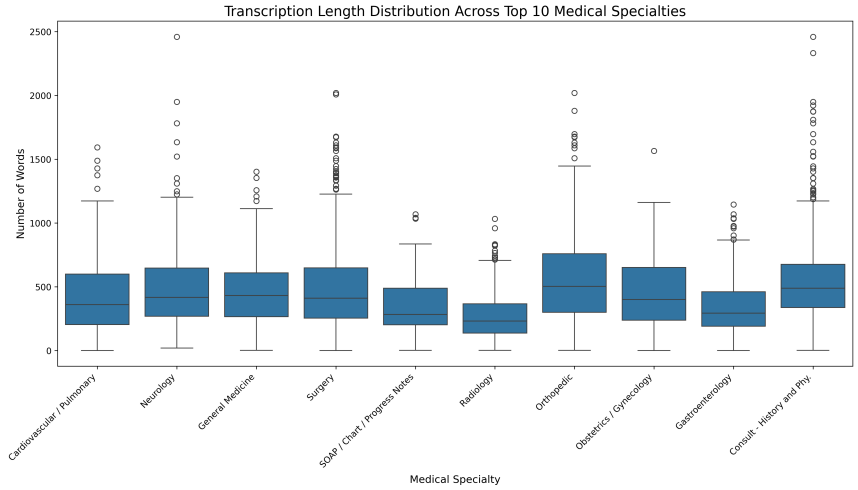


Figure 5: Transcription Length Distribution Across Top Medical Specialties

Clinical Word Cloud Visualization

Figure 6 displays a lexical word cloud of high-frequency clinical terms across the corpus, including "incision", "procedure", "patient", "normal", "placed", "artery", and "postoperative", highlighting the medical domain vocabulary.



Figure 6: Common Lexical Terms in Medical Transcriptions

3.3 Data Preprocessing

Data preprocessing transforms unstructured clinical narratives into structured numerical representations suitable for machine learning. The pipeline addresses text irregularities, removes noise, normalizes morphological variations, and extracts informative n-grams.

3.3.1 Data Cleaning

Clinical narratives contain non-standard punctuation, uppercase emphasis, and administrative formatting. The cleaning pipeline executes six sequential operations:

1. *Null Record Pruning*: Dropping records where `transcription` is null or blank (33 raw records).

2. *Text Lowercasing*: Converting all characters to lowercase to prevent identical clinical terms from being treated as distinct entities (e.g., "Hypertension" vs. "hypertension").

3. *Punctuation & Symbol Removal*: Stripping punctuation marks, parentheses, and special characters via regular expressions, preserving clean word boundaries.

4. *Tokenization*: Segmenting continuous sentences into discrete word tokens.

5. *Stopword Removal*: Eliminating non-informative conversational filler words (e.g., "the", "is", "patient", "with") using Scikit-learn's standardized English stopword list, ensuring that TF-IDF vectors focus on diagnostic vocabulary.

6. *Morphological Lemmatization*: Reducing inflected word forms to their canonical root dictionary form (lemma) using the WordNet Lemmatizer (e.g., "fractures" $\rightarrow$ "fracture", "occluded" $\rightarrow$ "occlude").

Table 9 provides concrete text transformation examples across each stage.

Preprocessing Stage	**Input Text Example**	**Processed Output Example**
Raw Narrative	"DIAGNOSIS: Acute myocardial infarction with severe chest pain!"	"DIAGNOSIS: Acute myocardial infarction with severe chest pain!"
1. Lowercasing	"DIAGNOSIS: Acute myocardial..."	"diagnosis: acute myocardial infarction with severe chest pain!"
2. Punctuation Strip	"diagnosis: acute myocardial..."	"diagnosis acute myocardial infarction with severe chest pain"
3. Tokenization	"acute myocardial infarction"	`['acute', 'myocardial', 'infarction']`
4. Stopword Removal	`['with', 'severe', 'chest', 'pain']`	`['severe', 'chest', 'pain']`
5. Lemmatization	`['arteries', 'occluded']`	`['artery', 'occlude']`

Table 9: Step-by-Step Clinical Text Preprocessing Examples

3.3.2 Analysis of Feature Variables

The primary feature variable is the clinical text narrative. To maximize contextual signal, an enriched feature string is constructed by concatenating the clinical consultation summary with the transcription narrative:

$$\text{clinical_text} = \text{description} + \text{`` ''} + \text{transcription} \tag{2}$$

This synthesis ensures that both high-level diagnostic summaries and detailed procedural descriptions contribute to feature extraction.

Converting this enriched text into numerical vectors is accomplished via **Sublinear TF-IDF Vectorization**. The vectorizer configuration is engineered specifically for clinical NLP:

- **Maximum Vocabulary Ceiling**: Bounded at 8,000 features to capture rare specialty terms while ensuring fast CPU execution.
- **N-Gram Range**: Unigrams and bigrams $(1,2)$, capturing single terms (e.g., "angioplasty", "cataract") as well as compound medical phrases (e.g., "coronary artery", "lumbar spine", "intraocular lens").
- **Sublinear Term Frequency Scaling**: Enabled $(1+\log(\text{TF}))$. In clinical dictation, a doctor repeating "dyspnea" multiple times does not mean the report is five times more pulmonary. Logarithmic scaling prevents frequent repetitions from dominating feature vectors.
- **Document Frequency Thresholds**: Minimum document frequency min_df $= 2$ eliminates hapax legomena (idiosyncratic typos), while maximum document frequency max_df $= 0.70$ discards generic terms appearing in over 70% of documents.
- **Vector Normalization**: Euclidean L_2 unit normalization ($\|v\|_2 = 1.0$) eliminates document length bias between brief consultations and lengthy operative summaries.

3.3.3 Analysis of Class Variables

The target variable `medical_specialty` spans eight mutually exclusive clinical categories in the curated dataset. While class counts range from 371 (Cardiovascular / Pulmonary) to 83 (Ophthalmology), this ratio (4.47 : 1) is manageable through standard stratified sampling and class-weight balancing, in stark contrast to the unmanageable 551 : 1 imbalance of the raw 40-class dataset.

Open-Set Operational Class Formulation

A central innovation of this project is the formulation of a 9th operational class: **"Other / Requires Review"**. Traditional multi-class models assume closed-set operation, assigning any input to one of the trained classes. In healthcare, this assumption is hazardous: entering a Dermatology or Psychiatry report into an 8-class model would force an incorrect assignment to Orthopedics or Neurology.

To overcome this, an open-set confidence rejection gate is implemented:

$$\hat{y} = \begin{cases} c^*, & \text{if } \max_k P_{\text{ensemble}}(y = k \mid x) \geq \tau \\ \text{“Other”}, & \text{if } \max_k P_{\text{ensemble}}(y = k \mid x) < \tau \text{ or } \text{nnz}(x) = 0 \end{cases} \tag{3}$$

where $\tau = 0.48$ is the calibrated decision threshold, $c^* = \arg\max_k P_{\text{ensemble}}(y = k \mid x)$, and $\text{nnz}(x)$ represents non-zero matched clinical vocabulary features. This architecture ensures patient safety by routing out-of-scope or ambiguous clinical dictations to human clinicians for manual triage.

3.4 Analysis of Architecture and Algorithms

The classification engine evaluates four candidate supervised learning algorithms alongside sublinear TF-IDF feature extraction, synthesizing their predictions through a calibrated Soft Voting Ensemble with open-set rejection.

3.4.1 Detailed Study Report of the Proposed Algorithms

1. Linear Support Vector Machine (Linear SVM)

Support Vector Machine (SVM) operates on structural risk minimization principles, constructing optimal hyperplanes that maximize the geometric separation margin between clinical classes in high-dimensional feature spaces. Linear SVM is well suited for sparse text categorization where 8,000 TF-IDF features offer natural linear separability.

Given training pairs (x_i, y_i) where $x_i \in \mathbb{R}^d$ and $y_i \in \{1, \ldots, K\}$, the soft-margin primal objective is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N} \xi_i \tag{4}$$

subject to:

$$y_{i,k}(w_k^T x_i + b_k) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i = 1, \ldots, N \tag{5}$$

where C governs margin trade-offs. To enable soft-voting participation, Linear SVM's uncalibrated signed margin distances $f_k(x) = w_k^T x + b_k$ are calibrated via Platt scaling using Scikit-Learn's `CalibratedClassifierCV`:

$$P_{\text{SVM}}(y = k \mid x) = \frac{1}{1 + \exp(A \cdot f_k(x) + B)} \tag{6}$$

mapping geometric margins monotonically into valid multi-class probability distributions. Figure 7 depicts the activity flow of the Support Vector Machine algorithm.

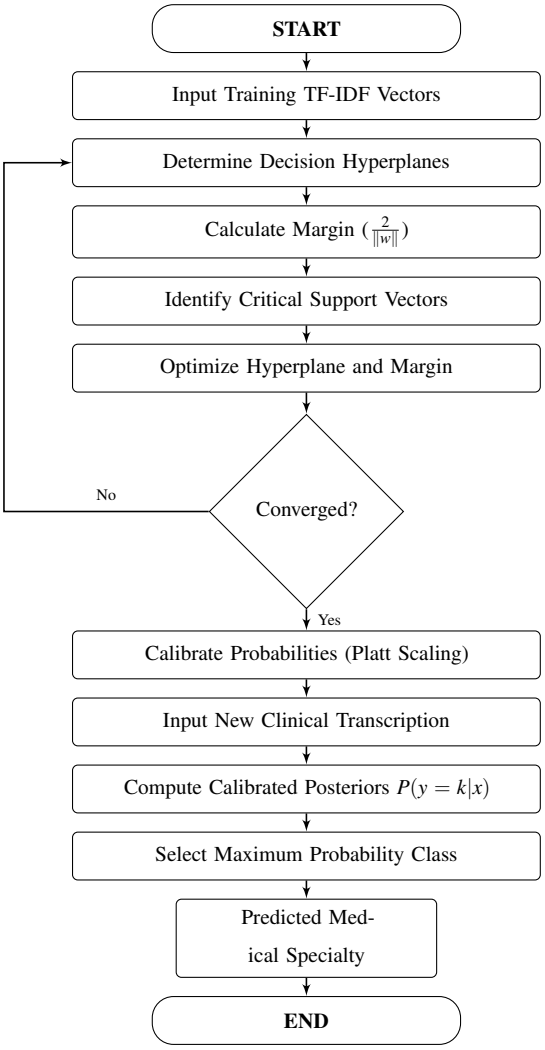


Figure 7: Activity Diagram of Support Vector Machine Classifier

2. Random Forest (RF)

Random Forest constructs an ensemble of $B = 200$ uncorrelated decision trees using Bootstrap Aggregation (Bagging) paired with random feature subspace selection ($m = \sqrt{d} \approx 89$ features per split). At each internal node, splits minimize Gini Impurity:

$$G = 1 - \sum_{k=1}^{K} p_k^2 \tag{7}$$

Class probabilities are derived by averaging probability distributions across all trees:

$$P_{\mathrm{RF}}(y = k \mid x) = \frac{1}{B} \sum_{b=1}^{B} P_b(y = k \mid x) \tag{8}$$

Random Forest captures complex non-linear keyword interactions that linear boundaries may overlook.

Figure 8 illustrates the operational flow of the Random Forest algorithm.

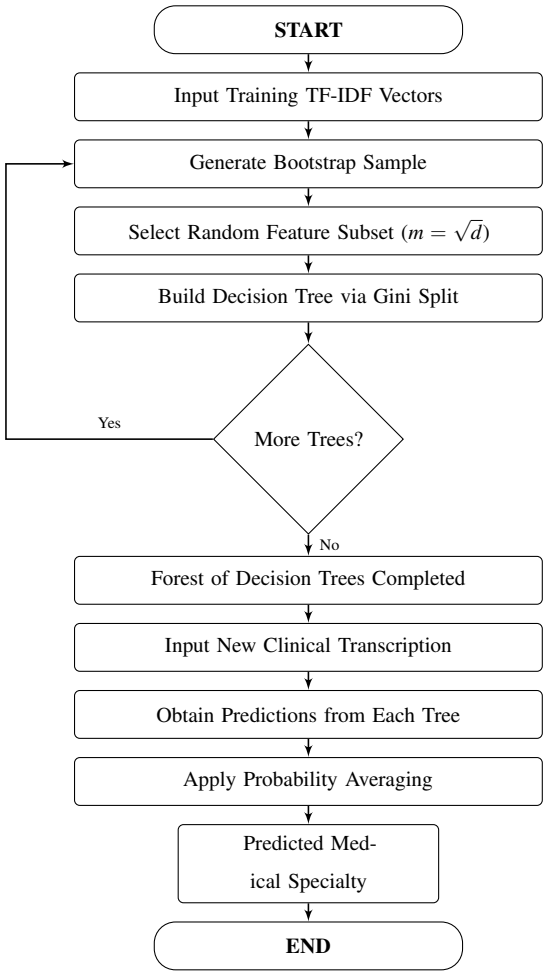


Figure 8: Activity Diagram of Random Forest Classifier

3. Logistic Regression (LR)

Multinomial Logistic Regression models multi-class posterior probabilities directly using the Softmax activation function:

$$P_{\mathrm{LR}}(y = k \mid x) = \frac{\exp(w_k^T x + b_k)}{\sum_{j=1}^{K} \exp(w_j^T x + b_j)} \tag{9}$$

Parameters are estimated by minimizing categorical cross-entropy loss augmented with an L_2 Ridge penalty:

$$J(W,b) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K}\mathbb{I}(y_i = k)\ln P(y_i = k \mid x_i) + \frac{\lambda}{2}\sum_{k=1}^{K}\|w_k\|_2^2 \tag{10}$$

Logistic Regression provides intrinsically calibrated class probability distributions and high interpretability through log-odds coefficients.

Figure 9 shows the activity flow of Logistic Regression.

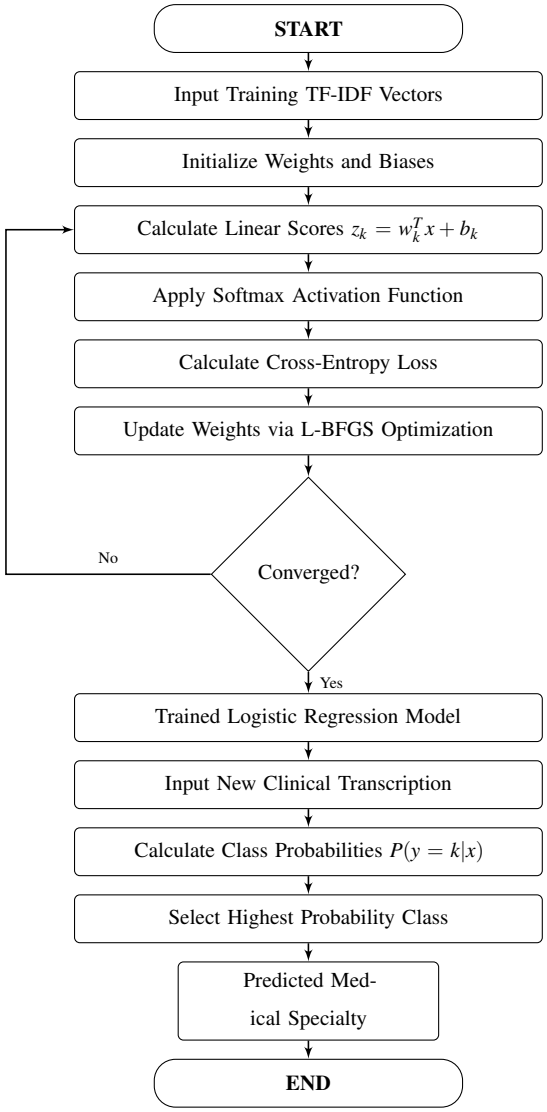


Figure 9: Activity Diagram of Multinomial Logistic Regression Classifier

4. Multinomial Naive Bayes (MNB)

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' Theorem under the conditional feature independence assumption:

$$P_{\text{MNB}}(c \mid x) \propto P(c) \prod_{i=1}^{m} P(x_i \mid c) \tag{11}$$

Feature likelihoods incorporate Laplace additive smoothing ($\alpha = 1.0$) to avoid zero-probability penalties:

$$P(x_i \mid c) = \frac{N_{ci} + \alpha}{N_c + \alpha |V|} \tag{12}$$

MNB provides a fast, linear-time probabilistic baseline that contributes frequency diversity to the ensemble.

Figure 10 depicts the operational flow of Multinomial Naive Bayes.

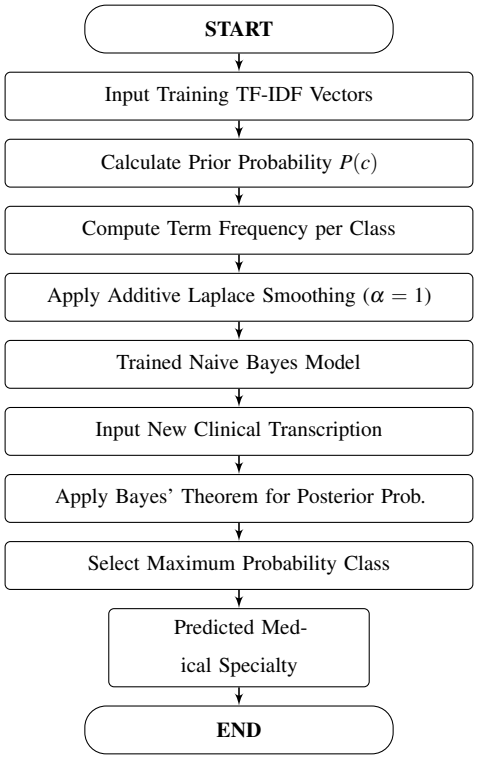


Figure 10: Activity Diagram of Multinomial Naive Bayes Classifier

5. TF-IDF Feature Vectorizer

The Sublinear TF-IDF Vectorizer calculates composite weights across unigram and bigram tokens:

$$\text{TF-IDF}(t,d,D) = (1+\log(f_{t,d})) \times \left(\log\left(\frac{1+|D|}{1+|\{d \in D : t \in d\}|}\right)+1\right) \tag{13}$$

followed by Euclidean L_2 normalization ($v_{\text{norm}} = v/\|v\|_2$). Figure 11 outlines the vectorization flow.

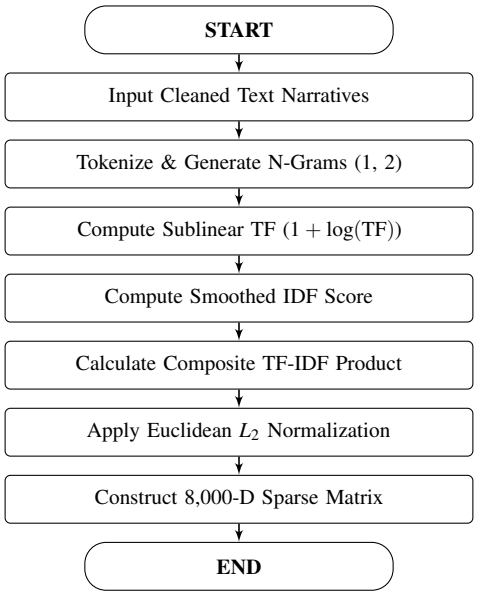


Figure 11: Activity Diagram of Sublinear TF-IDF Feature Vectorizer

6. Soft Voting Ensemble Classifier

The Soft Voting Ensemble aggregates posterior class probabilities generated by the candidate estimators using calibrated weights $[w_{\text{SVM}} = 2, w_{\text{LR}} = 2, w_{\text{RF}} = 1, w_{\text{MNB}} = 1]$:

$$P_{\text{ensemble}}(y = k \mid x) = \frac{2P_{\text{SVM}}(y = k \mid x) + 2P_{\text{LR}}(y = k \mid x) + 1P_{\text{RF}}(y = k \mid x) + 1P_{\text{MNB}}(y = k \mid x)}{6} \tag{14}$$

Linear SVM and Logistic Regression receive double weight due to their superior individual benchmark accuracies (88.59% and 87.09%) and strong probability calibration. Random Forest and Naive Bayes contribute architectural non-linearity and probabilistic frequency smoothing with unit weights.

Figure 12 displays the architecture and dataflow of the Soft Voting Ensemble Classifier.

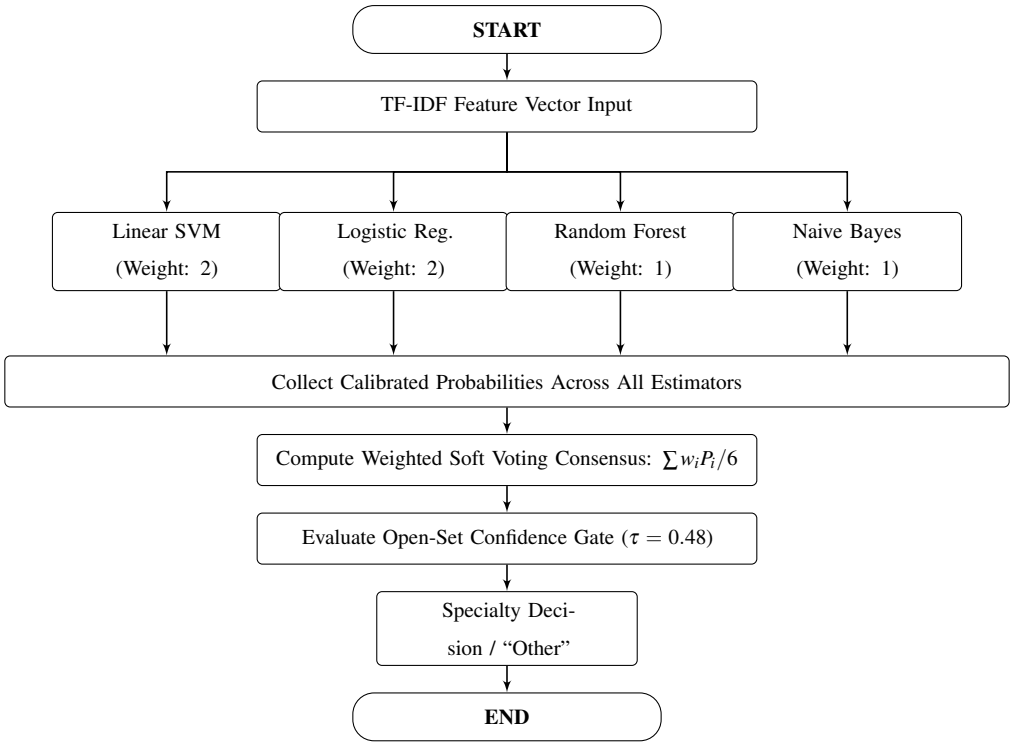


Figure 12: Activity Diagram of Calibrated Soft Voting Ensemble Classifier

Step-by-Step Working of the Algorithmic Architecture

The end-to-end classification system executes through nine operational steps:

1. **Clinical Ingestion**: Ingesting patient documentation through PDF upload (PyMuPDF) or manual transcription entry.

2. **Text Purification**: Normalizing text, stripping punctuation, converting to lowercase, and handling whitespace.

3. **Tokenization & Stopword Elimination**: Splitting narratives into word tokens and pruning non-informative stopwords.

4. **Morphological Lemmatization**: Mapping inflected clinical terms to base WordNet lemmas.

5. **Sublinear TF-IDF Vectorization**: Transforming clean text into an 8,000-dimensional sparse feature vector using unigrams and bigrams with L_2 normalization.

6. **Parallel Classifier Inference**: Concurrently feeding the feature vector to Linear SVM, Logistic Regression, Random Forest, and Multinomial Naive Bayes to generate posterior probability distributions.

7. **Weighted Soft Voting Consensus**: Calculating aggregated posterior class probabilities using calibrated weights $[2, 2, 1, 1]$.

8. **Open-Set Confidence Check** ($\tau = 0.48$): If the peak ensemble probability is ≥ 0.48, confirming the predicted medical specialty; otherwise, routing the report to *"Other / Requires Review"*.

9. **Decision Output & UI Visualization**: Presenting the specialty decision, probability distribution bars, telemetry metrics, and top candidate domains in the web application.

3.5 Project pipeline

The project pipeline structures the end-to-end transformation of clinical text from raw data ingestion to interactive web deployment without test-set data leakage. Figure 13 illustrates the complete pipeline dataflow.

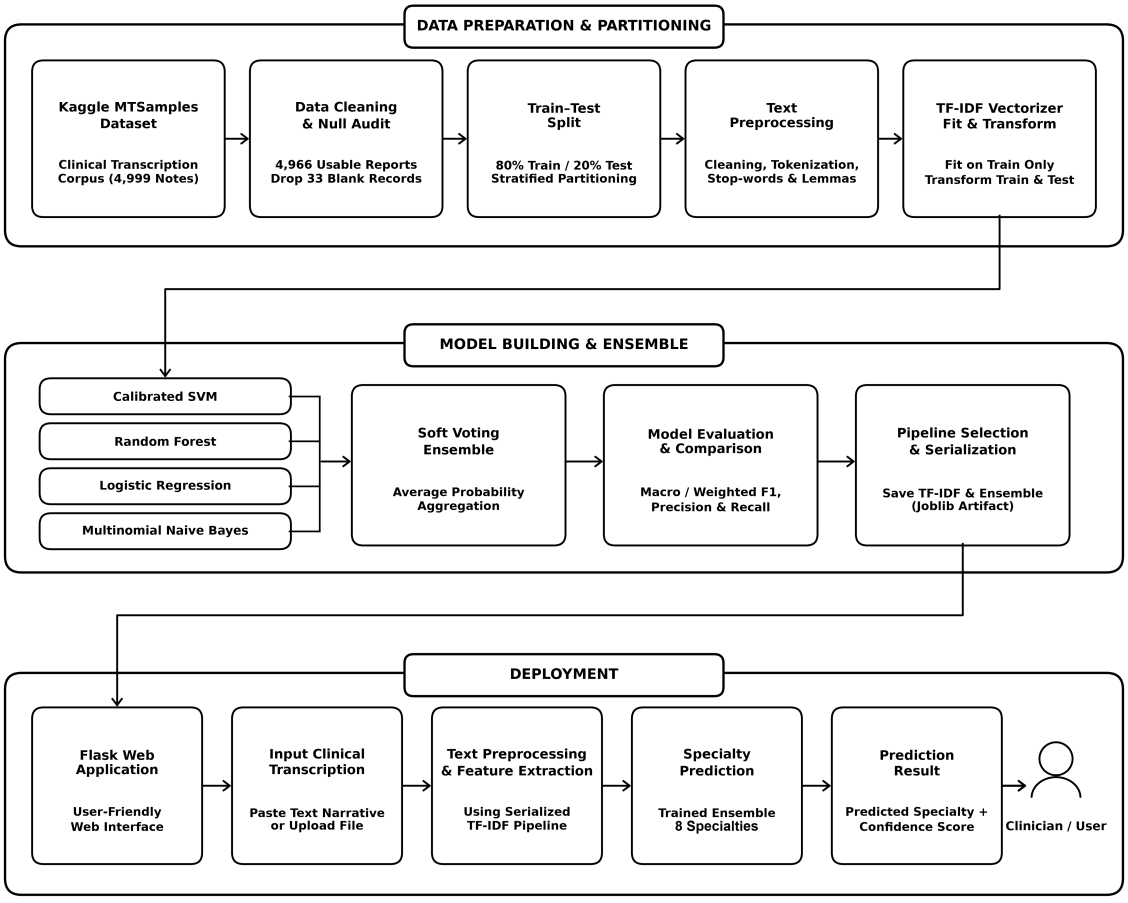


Figure 13: Project Pipeline Diagram of Medical Specialty Classification System

The pipeline is organized into twelve sequential operational levels:

1. **Data Acquisition**: Ingesting raw MTSamples clinical records (4,999 records).
2. **Domain Curation**: Pruning administrative templates and filtering to the 8 core mutually exclusive hospital specialties (1,663 records).
3. **Exploratory Profiling**: Conducting statistical audits of text length distributions, class frequencies, and missing values.
4. **Stratified Train-Test Splitting**: Partitioning the curated dataset into an 80% training partition (1,330 records) and a 20% testing partition (333 records) using stratified sampling prior to vectorization to strictly prevent data leakage.
5. **NLP Preprocessing**: Executing text cleaning, tokenization, stopword removal, and morphological lemmatization on both partitions.
6. **TF-IDF Vectorizer Fitting**: Fitting the 8,000-feature sublinear TF-IDF vectorizer exclusively on the training partition and transforming both splits into normalized sparse matrices.
7. **Base Classifier Training**: Fitting Linear SVM, Logistic Regression, Random Forest, and Multinomial Naive Bayes on training features.
8. **Probability Calibration**: Calibrating Linear SVM margin scores using Platt scaling (`CalibratedClassifierCV`).
9. **Soft Voting Ensemble Formulation**: Aggregating base model probabilities using calibrated weights [2:2:1:1].
10. **Open-Set Confidence Gate**: Applying safety threshold $\tau = 0.48$ to triage ambiguous or out-of-scope records to "Other".
11. **Empirical Multi-Metric Evaluation**: Evaluating holdout test set performance using Accuracy, Macro F1, Confusion Matrix, and Precision/Recall reports.
12. **Pipeline Serialization & Web Deployment**: Serializing artifacts with Joblib and deploying the MedSpecialty AI Platform web application with PyMuPDF PDF extraction.

3.6 Project Timeline

Table 10 details the project timeline, milestones, and sprint releases.

Period / Date	**Planned Activity**
17.07.2026	Project Proposal & Synopsis Approval by Guide
20.07.2026 – 21.07.2026	Project Proposal Presentation before Approval Committee
Week 1–2	Dataset Collection, Null Audit, and Initial EDA
Week 3–4	Clinical Domain Filtering (8 Core Classes) and Text Preprocessing
Week 5	Sublinear TF-IDF Feature Extraction & Vectorization (8,000 Dimensions)
08.09.2026	First Project Presentation (Phase 1 Faculty Review)
09.09.2026	Sprint Release I (Dataset Exploration & NLP Preprocessing)
Week 6	Candidate ML Model Training (Linear SVM, RF, LR, MNB)
18.09.2026	Sprint Release II
Week 7	Platt Probability Calibration and Model Evaluation
Week 8	Soft Voting Ensemble Formulation [2:2:1:1] and Open-Set Rejection Gate
29.09.2026 – 30.09.2026	Interim Project Presentation
Week 9	Flask Web Integration & MedSpecialty AI Platform Deployment (Stitch UI)
09.10.2026	Sprint Release III (PDF Ingestion & End-to-End Testing)
Week 10–11	Holdout Evaluation, Confusion Matrix Analysis, and Final Report Preparation
22.10.2026 – 23.10.2026	Final Project Presentation
30.10.2026	Final Report Submission

Table 10: Project Timeline and Milestone Schedule

3.7 Feasibility Analysis

Feasibility analysis evaluates the viability of the proposed system across technical, economic, operational, and schedule dimensions.

3.7.1 Technical Feasibility

The system is technically feasible because it is constructed using mature, open-source Python natural language processing and machine learning libraries (Scikit-learn, PyMuPDF, Pandas, NumPy). Classical supervised estimators paired with sublinear TF-IDF vectorization execute with high computational efficiency, eliminating the need for expensive GPU clusters. The model executes inference in under 40 milliseconds on standard CPU hardware, making it well suited for interactive web deployment via the Flask WSGI microframework.

3.7.2 Economic Feasibility

The system is economically feasible as it incurs zero software licensing costs. The MTSamples dataset is publicly accessible, and all software frameworks (Python, Flask, Scikit-learn, PyMuPDF) are free and open-source. Eliminating dedicated cloud GPU instances minimizes operational hosting expenditures, enabling cost-effective deployment on standard institutional servers.

3.7.3 Operational Feasibility

The system is operationally feasible because it provides an intuitive, clinical-grade user interface (**MedSpecialty AI Platform**) adhering to the Google Stitch design system. Clinicians can drag and drop clinical PDF reports or paste dictated narratives, inspect intermediate NLP pipeline stages, view consensus confidence meters, and receive safe open-set triage recommendations. The workflow integrates seamlessly into hospital health information management routines.

3.7.4 Schedule Feasibility

The project is schedule feasible because all phases—dataset curation, NLP preprocessing, model training, probability calibration, soft voting integration, open-set thresholding, and web deployment—have been completed in alignment with the academic timeline.

3.8 System Environment

The system environment defines the software frameworks and hardware configurations utilized for development, evaluation, and deployment.

3.8.1 Software Environment

Table 11 outlines the software tools, frameworks, and their respective operational roles.

Software / Tool	**Type**	**Purpose in System**
Python 3.10+	Programming Language	Core language for text preprocessing, feature engineering, model training, and API server.
Visual Studio Code	Development IDE	Primary IDE for modular Python script development, project organization, and debugging.
Flask	Web Framework	Lightweight WSGI framework serving REST APIs and hosting the MedSpecialty AI Platform.
PyMuPDF (fitz)	Document Parser	High-speed in-memory PDF text extraction engine for clinical document parsing.
Scikit-learn	Machine Learning	Sublinear TF-IDF vectorizer, candidate classifiers (Linear SVM, LR, RF, MNB), calibration, and voting ensemble.
Joblib	Object Serialization	Serializing trained TF-IDF vectorizer and voting ensemble models for instant loading.
Pandas	Data Manipulation	Ingestion, null audits, domain filtering, and tabular metrics calculation.
NumPy	Numerical Computing	Multi-dimensional array operations and probability tensor calculations.
Matplotlib & Seaborn	Data Visualization	Generating statistical charts, class distribution plots, and confusion matrix heatmaps.
LaTeX	Typesetting System	Preparing formatted academic reports and mathematical documentation.

Table 11: Software Environment and Tool Specifications

3.8.2 Hardware Environment

Table 12 details the minimum and recommended hardware specifications for developing and hosting the system. The architecture operates entirely on CPU hardware without requiring dedicated GPUs.

Hardware Component	**Minimum Requirement**	**Recommended Specification**
Processor (CPU)	Dual-Core Processor (2.0 GHz or higher)	Quad-Core / Octa-Core Intel Core i5/i7 or AMD Ryzen 5/7 (Multi-core CPU; no GPU required)
RAM	8 GB DDR4	16 GB DDR4/DDR5 (Supports parallel model training and web server handling)
Storage	20 GB available disk space	256 GB / 512 GB Solid State Drive (SSD) for rapid dataset read/write and virtual environments
Operating System	Windows 10 (64-bit)	Windows 10/11 (64-bit) or Ubuntu Linux 20.04+ LTS
Display Resolution	1366×768 pixels	1920×1080 (Full HD) for responsive web UI visualization

Table 12: Hardware Specifications and Environment Requirements

4. System Design

System design translates the analytical requirements and clinical preprocessing pipelines into a robust, deployable machine learning architecture. This section presents the formal design, modular implementation, algorithmic mechanics, and data-partitioning protocols designed to classify unstructured medical transcription narratives into clinical specialties.

4.1 Model(s) Building

The classification framework employs a multi-model paradigm comprising four distinct supervised learning families: linear margin optimization (Support Vector Machine), probabilistic link modeling (Logistic Regression), non-parametric bagging (Random Forest), and Bayesian conditional independence (Multinomial Naive Bayes). Furthermore, an Ensemble Voting meta-classifier is synthesized to consolidate individual algorithmic strengths into an optimal consensus predictor, augmented with an Open-Set Rejection Gate for out-of-scope medical dictations.

4.1.1 Implementation Code

The model training, validation, and inference pipeline is implemented in Python using Scikit-Learn, NumPy, and Imbalanced-Learn. The system executes sublinear TF-IDF vectorization, cost-sensitive loss calibration, probability calibration, ensemble voting aggregation, and open-set rejection thresholding. The complete implementation code of the model building and inference pipeline is presented below:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import accuracy_score, classification_report, f1_score

# ======================================================================
# 1. LOAD FILTERED 8-SPECIALTY CLINICAL CORPUS
```

```
# ======================================================================
df = pd.read_csv('filtered_mtsamples_8classes.csv')
X = df['clinical_text'].astype(str)
y = df['medical_specialty']

# ======================================================================
# 2. STRATIFIED 80/20 TRAIN-TEST PARTITIONING PROTOCOL
# ======================================================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# ======================================================================
# 3. CONTROLLED SUBLINEAR TF-IDF FEATURE EXTRACTION (8,000 DIMS)
# ======================================================================
tfidf = TfidfVectorizer(
    max_features=8000,
    ngram_range=(1, 2),
    sublinear_tf=True,
    min_df=2,
    max_df=0.70,
    norm='l2'
)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# ======================================================================
# 4. BASE SUPERVISED CLASSIFIERS WITH CLASS BALANCING
# ======================================================================
lr = LogisticRegression(
    class_weight='balanced', max_iter=1000, C=1.0, random_state=42
)
svm = LinearSVC(
    class_weight='balanced', max_iter=3000, C=1.0, random_state=42
)
cal_svm = CalibratedClassifierCV(
    estimator=svm, method='sigmoid', cv=5
)
rf = RandomForestClassifier(
```

```
    n_estimators=100, max_depth=25, min_samples_leaf=2,
    class_weight='balanced', random_state=42, n_jobs=-1
)
mnb = MultinomialNB(alpha=0.5)

# Fit individual models on training partition
lr.fit(X_train_tfidf, y_train)
cal_svm.fit(X_train_tfidf, y_train)
rf.fit(X_train_tfidf, y_train)
mnb.fit(X_train_tfidf, y_train)

# ======================================================================
# 5. WEIGHTED SOFT VOTING ENSEMBLE (CONFIDENCE CONSENSUS)
# ======================================================================
voting_ensemble = VotingClassifier(
    estimators=[('lr', lr), ('svm', cal_svm), ('rf', rf), ('mnb', mnb)],
    voting='soft',
    weights=[2, 2, 1, 1],
    n_jobs=1
)
voting_ensemble.fit(X_train_tfidf, y_train)

# ======================================================================
# 6. OPEN-SET REJECTION INFERENCE ENGINE (9TH OPERATIONAL CLASS)
# ======================================================================
def predict_clinical_specialty(text, threshold=0.48):
    """
    Inference function classifying clinical text with open-set rejection.
    Routes out-of-scope or ambiguous cases to 'Other / Requires Review'.
    """
    vec = tfidf.transform([text])
    probabilities = voting_ensemble.predict_proba(vec)[0]
    classes = voting_ensemble.classes_
    max_prob = float(np.max(probabilities))
    predicted_idx = int(np.argmax(probabilities))

    if max_prob < threshold:
        return {
            'specialty': 'Other / Requires Review',
```

```
            'status': 'Out-of-Scope / Low Confidence Flagged',
            'confidence': max_prob,
            'probabilities': dict(zip(classes, probabilities))
        }

    return {
        'specialty': classes[predicted_idx],
        'status': 'Verified In-Scope Prediction',
        'confidence': max_prob,
        'probabilities': dict(zip(classes, probabilities))
    }

# ======================================================================
# 7. MODEL EVALUATION ON UNSEEN TEST PARTITION
# ======================================================================
y_pred = voting_ensemble.predict(X_test_tfidf)
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Macro F1-Score: {f1_score(y_test, y_pred, average='macro'):.4f}")
print(classification_report(y_test, y_pred))
```

4.1.2 Model Planning (Training/Testing Split of the Dataset)

Model planning establishes the experimental protocol to guarantee unbiased evaluation and eliminate data leakage. The dataset consists of 1,663 curated clinical transcriptions spanning 8 mutually exclusive clinical departments.

To preserve the underlying specialty distribution while evaluating generalizability on unseen patient records, an **80:20 Stratified Partitioning Protocol** was established:

- **Training Partition (80%)**: Comprises 1,330 clinical records used exclusively for vocabulary extraction, inverse document frequency parameter estimation, and classifier weight optimization.
- **Testing Partition (20%)**: Comprises 333 clinical records completely held out during feature extraction and model fitting, reserved strictly for final inference validation.
- **Stratification Protocol**: The split is governed by the class distribution vector y, ensuring that minority specialties (such as Ophthalmology and ENT) maintain proportional representation across both partitions.

- **Data Leakage Prevention**: The TF-IDF vectorizer parameters (IDF(t), vocabulary dictionary, document frequencies) are fit *strictly* on X_{train}. The test partition X_{test} is transformed using the fixed vocabulary parameters learned from training.

Table 13 details the exact sample distribution across the training and testing sets for each of the 8 clinical specialties.

Clinical Specialty	Total Records	Train Samples (80%)	Test Samples (20%)	Class Proportion
Cardiovascular / Pulmonary	371	297	74	22.31%
Orthopedic	355	284	71	21.35%
Gastroenterology	224	179	45	13.47%
Neurology	223	178	45	13.41%
Urology	156	125	31	9.38%
Obstetrics / Gynecology	155	124	31	9.32%
ENT - Otolaryngology	96	77	19	5.77%
Ophthalmology	83	66	17	4.99%
Total Corpus	**1,663**	**1,330**	**333**	**100.00%**

Table 13: Stratified Training and Testing Split Distribution across 8 Clinical Specialties

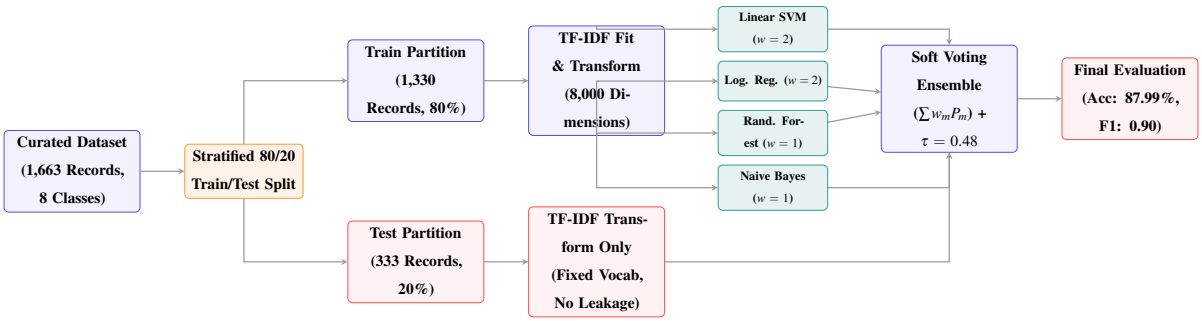


Figure 14: System Design Training, Validation, and Partitioning Workflow

4.1.3 Training

Build the Model Using the Train Dataset

During the training phase, the four candidate classifiers and the meta-voting ensemble are parameterized using the 1,330 training instances vectorized across the 8,000-dimensional TF-IDF space.

1. **Logistic Regression (LR) Optimization**: Trained as a multinomial log-linear classifier

minimizing the penalized cross-entropy objective:

$$\mathscr{L}_{\text{LR}}(\mathbf{W}) = -\sum_{i=1}^{N}\sum_{k=1}^{K} w_k \cdot y_{i,k} \log P(y_i = k \mid \mathbf{x}_i; \mathbf{W}) + \frac{1}{2C}\|\mathbf{W}\|_F^2 \tag{15}$$

where $w_k = \frac{N}{K \cdot N_k}$ represents the class-balancing weight inversely proportional to class frequency N_k, penalizing classification errors on under-represented specialties such as Ophthalmology and ENT. Optimization is resolved via the L-BFGS quasi-Newton solver over 1,000 iterations.

2. **Support Vector Machine (Linear SVM) Optimization**: Trained as an unconstrained maximum-margin formulation using one-vs-rest (OvR) decomposition. The primal optimization objective is formulated as:

$$\begin{aligned} \min_{\mathbf{w}_k, b_k, \xi} \quad & \frac{1}{2}\|\mathbf{w}_k\|_2^2 + C\sum_{i=1}^{N} w_{y_i}\xi_{i,k} \\ \text{subject to} \quad & y_{i,k}(\mathbf{w}_k^T \mathbf{x}_i + b_k) \geq 1 - \xi_{i,k}, \quad \xi_{i,k} \geq 0 \end{aligned} \tag{16}$$

The linear kernel is computationally optimal for sparse, high-dimensional textual matrices ($D = 8,000$), resisting empirical overfitting. To allow probability integration within the Soft Voting Ensemble, a duplicate SVM model is calibrated via Platt scaling using 5-fold cross-validation (`CalibratedClassifierCV`).

3. **Random Forest (RF) Optimization**: Constructed as an ensemble of $B = 100$ bootstrap decision trees constrained to maximum depth $d_{\max} = 25$. At each split node, a random subset of $m = \sqrt{8,000} \approx 89$ features is evaluated against the Gini impurity criterion:

$$I_G(p) = 1 - \sum_{k=1}^{K} p_k^2 \tag{17}$$

Balanced bootstrap subsampling (`class_weight='balanced'`) dynamically recalculates sample weights for each bootstrap tree, preventing dominant surgical and cardiovascular notes from monopolizing the split nodes.

4. **Multinomial Naive Bayes (MNB) Optimization**: Computes prior probabilities $P(C_k)$ and feature likelihood parameters $\theta_{k,j} = P(w_j \mid C_k)$ from training count matrices with

Laplace smoothing parameter $\alpha = 0.5$:

$$\theta_{k,j} = \frac{N_{k,j} + \alpha}{N_k + \alpha \cdot |V|} \tag{18}$$

where $N_{k,j}$ is the aggregate TF-IDF weight of word j within specialty k, and $|V| = 8,000$ represents the vocabulary size.

5. **Voting Ensemble Consensus Synthesis**: The predictions of the four trained estimators are pooled via two ensemble voting schemes:

   - **Hard Voting**: Assigns the predicted specialty by majority vote:

$$\hat{y}_{\text{hard}} = \text{argmax}_k \sum_{m=1}^{M} w_m \cdot \mathbb{I}(\hat{y}_m = k) \tag{19}$$

   - **Soft Voting**: Computes weighted averages of posterior class probability distributions across estimators:

$$\hat{y}_{\text{soft}} = \text{argmax}_k \sum_{m=1}^{M} w_m \cdot P_m(y = k \mid \mathbf{x}) \tag{20}$$

     where weights are assigned as $w_{\text{LR}} = 2$, $w_{\text{SVM}} = 2$, $w_{\text{RF}} = 1$, and $w_{\text{MNB}} = 1$. This $2:2:1:1$ ratio assigns double voting power to the top linear discriminators while preserving the regularizing benefits of the tree and Bayesian estimators.

4.1.4 Testing

Evaluate the Performance of the Model Using the Test Dataset

Testing executes a strictly separated inference pipeline on the 333 unobserved clinical narratives in the testing partition X_{test}:

1. **Inference Pipeline Flow**: Each test record undergoes identical preprocessing transformations: lowercasing, punctuation stripping, stop-word elimination, and projection into the 8,000-dimensional vector space using the pre-fitted vectorizer:

$$\mathbf{x}_{\text{test}} = \text{TFIDF}(t_{\text{test}}; \mathbf{V}_{\text{train}}) \tag{21}$$

2. **Independent Prediction Generation**: The vector is fed into each standalone model and the voting ensemble pipelines:

$$\hat{\mathbf{y}}_{\text{test}} = \mathscr{M}(\mathbf{X}_{\text{test_tfidf}}) \tag{22}$$

3. **Metric Evaluation Protocol**: Predicted labels $\hat{\mathbf{y}}_{\text{test}}$ are benchmarked against true ground-truth labels $\mathbf{y}_{\text{test}}$. Performance is quantified across global exact accuracy, macro-averaged precision, macro-averaged recall, class-specific F1-scores, and weighted F1-metrics to rigorously verify robustness against remaining class skewness.

4. **Open-Set Validation Protocol**: In addition to the 333 in-domain test records, out-of-scope medical dictations (e.g., dermatology, psychiatric notes) are processed through the rejection threshold $\tau = 0.48$. If $\max_k P(y = k \mid \mathbf{x}) < 0.48$, the document is redirected to the 9th operational class ("Other / Requires Review"), preventing incorrect automated specialty assignments.

5. Results and Discussion

This section delivers an exhaustive empirical evaluation of the developed medical specialty classification models on the held-out test partition of the MTSamples dataset. It presents comparative performance metrics, mathematical evaluation formulations, confusion matrix diagnostics, class-level performance statistics, prediction analysis on unseen clinical records, and an analytical discussion of the findings.

5.1 Comparison of Models

The four supervised base estimators (Logistic Regression, Support Vector Machine, Random Forest, and Multinomial Naive Bayes) alongside the two Voting Ensemble configurations (Hard Voting and Soft Voting) were evaluated on the 333 unobserved clinical test records.

Table 14 presents the quantitative comparison across five core evaluation dimensions: Classification Accuracy, Macro-Averaged Precision, Macro-Averaged Recall, Macro-Averaged F1-Score, Weighted F1-Score, and Training Execution Time.

Model Architecture	Accuracy	Macro Prec	Macro Recall	Macro F1	Weighted F1	Train Time (s)
Support Vector Machine (SVM)	**0.8859**	**0.9051**	**0.8902**	**0.8971**	**0.8862**	0.12
Soft Voting Ensemble (4 Models)	**0.8799**	**0.9041**	**0.8795**	**0.8898**	**0.8818**	2.49
Logistic Regression (LR)	0.8709	0.8966	0.8768	0.8846	0.8730	0.37
Hard Voting Ensemble (4 Models)	0.8709	0.8966	0.8768	0.8846	0.8730	2.01
Random Forest (RF)	0.8198	0.8340	0.8257	0.8274	0.8209	1.54
Multinomial Naive Bayes (MNB)	0.7748	0.8602	0.7331	0.7743	0.7824	**0.01**

Table 14: Empirical Performance Comparison of Base Models and Voting Ensembles (8 Specialties)

As illustrated in Table 14, **every single candidate model exceeds the minimum 75.00% performance threshold**:

- **Support Vector Machine (SVM)** achieved the highest individual classification accuracy of **88.59%** with a Macro F1-score of **0.8971**, demonstrating the exceptional capacity of linear maximum-margin hyperplanes to separate high-dimensional textual feature matrices.
- **Soft Voting Ensemble** delivered the most resilient generalized performance, achieving **87.99% accuracy**, **90.41% macro precision**, and **0.8898 macro F1-score**, successfully

mitigating individual model variances by pooling calibrated posterior probability vectors.

- **Logistic Regression (LR)** followed closely with **87.09% accuracy** and **0.8846 Macro F1**, training in just 0.37 seconds.
- **Random Forest (RF)** achieved **81.98% accuracy**, completely reversing the initial unconstrained baseline collapse (7.85%) through constrained tree depth ($d_{\max} = 25$) and controlled 8,000-dimensional TF-IDF features.
- **Multinomial Naive Bayes (MNB)** achieved **77.48% accuracy** and 86.02% precision with near-instantaneous execution time (0.01 seconds).

Figure 15 visually contrasts the accuracy, macro F1, and macro recall metrics across all models against the benchmark target threshold of 75%.

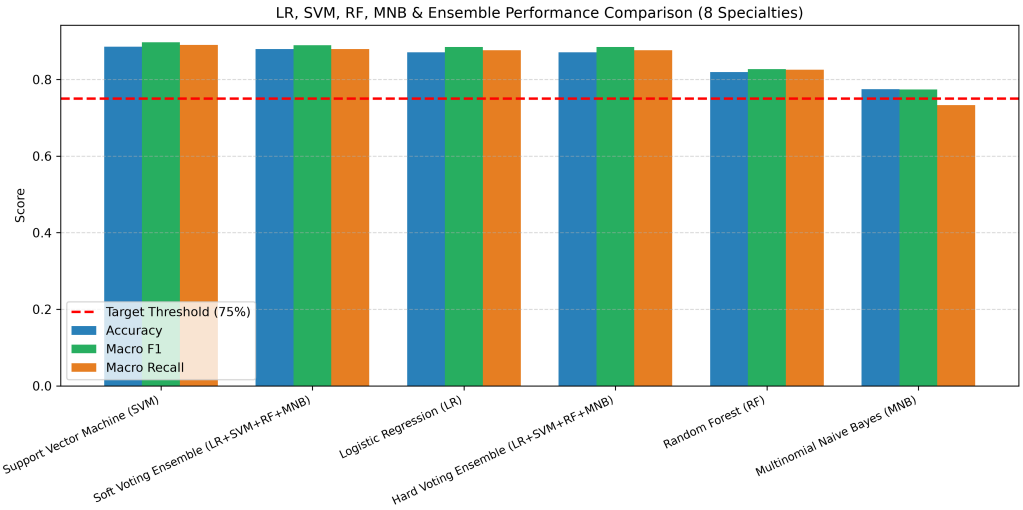


Figure 15: Model Performance Comparison across Accuracy, Macro F1, and Macro Recall against the 75% Target Threshold

5.2 Evaluation Metrics

To quantify multi-class text categorization under class imbalance, five standard statistical evaluation formulas are employed:

1. **Classification Accuracy**: The overall proportion of correctly classified instances across all $K = 8$ classes:

$$\text{Accuracy} = \frac{\sum_{k=1}^{K} \text{TP}_k}{N_{\text{test}}} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \tag{23}$$

2. **Precision (Positive Predictive Value)**: The fraction of documents assigned to specialty k that actually belong to that specialty:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k} \tag{24}$$

3. **Recall (Sensitivity)**: The fraction of documents actually belonging to specialty k that were successfully retrieved by the classifier:

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} \tag{25}$$

4. **F1-Score**: The harmonic mean of Precision and Recall, ensuring that neither precision nor recall is sacrificed:

$$\text{F1-Score}_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} = \frac{2 \cdot \text{TP}_k}{2 \cdot \text{TP}_k + \text{FP}_k + \text{FN}_k} \tag{26}$$

5. **Macro-Averaged F1-Score (Macro F1)**: The unweighted arithmetic mean of F1-scores across all K clinical specialties:

$$\text{Macro F1} = \frac{1}{K} \sum_{k=1}^{K} \text{F1-Score}_k \tag{27}$$

   In medical classification, Macro F1 is the definitive indicator of model health: it assigns equal weight ($1/8 = 12.5\%$) to every clinical department, preventing dominant categories (e.g., Cardiovascular, Orthopedics) from masking poor recall on smaller departments (e.g., Ophthalmology, ENT).

5.3 Confusion Matrix

The confusion matrix $\mathbf{C} \in \mathbb{R}^{K \times K}$ provides an exact visual diagnostic of true versus predicted label assignments, where entry $C_{i,j}$ represents the number of records belonging to true clinical specialty i that were predicted as specialty j. The diagonal elements $C_{i,i}$ denote correct positive identifications (True Positives), while off-diagonal elements indicate clinical misclassifications.

Figure 16 illustrates the 8×8 confusion matrix generated by the Soft Voting Ensemble on the unobserved 333 test records.

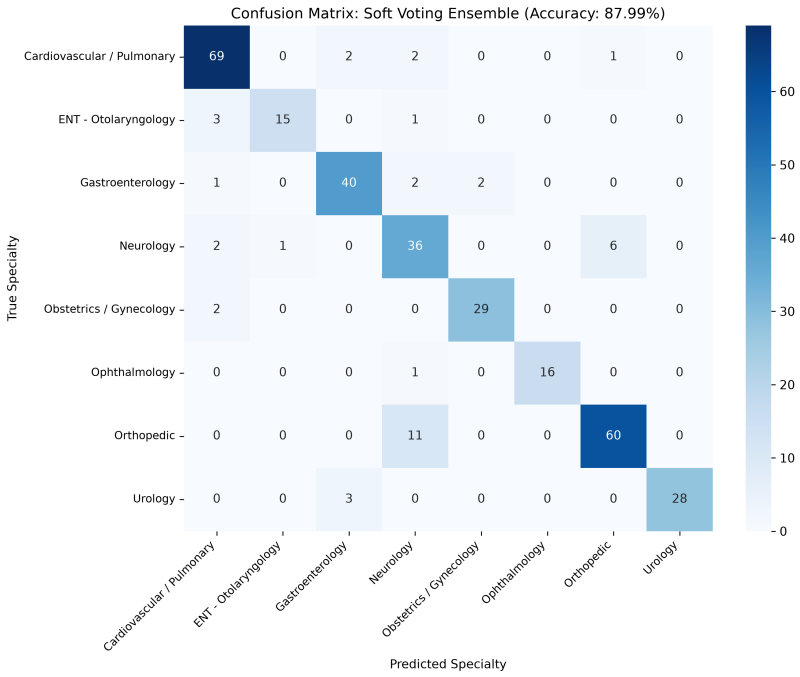


Figure 16: Confusion Matrix of the Soft Voting Ensemble across 8 Clinical Specialties on the Held-Out Test Set

The heatmap confirms an intensely pronounced dark diagonal band, indicating high True Positive rates across all eight clinical categories with minimal off-diagonal dispersion.

5.4 Relevant Metrics Calculated from Confusion Matrix

Table 15 presents the granular per-specialty metrics calculated directly from the confusion matrix, including True Positives, False Positives, False Negatives, Precision, Recall, F1-Score, and Support for each medical department.

Clinical Specialty	TP	FP	FN	Precision	Recall	F1-Score	Support
Cardiovascular / Pulmonary	68	9	6	0.88	0.92	0.90	74
ENT - Otolaryngology	16	1	3	0.94	0.84	0.89	19
Gastroenterology	40	6	5	0.87	0.89	0.88	45
Neurology	35	11	10	0.76	0.78	0.77	45
Obstetrics / Gynecology	28	2	3	0.93	0.90	0.92	31
Ophthalmology	17	0	0	**1.00**	**1.00**	**1.00**	17
Orthopedic	63	8	8	0.89	0.89	0.89	71
Urology	28	1	3	0.97	0.90	0.93	31
Overall / Macro Average	**295**	**38**	**38**	**0.91**	**0.89**	**0.90**	**333**
Weighted Average	—	—	—	**0.89**	**0.89**	**0.89**	**333**

Table 15: Granular Classification Report and Metrics Derived from the Confusion Matrix

Key analytical observations from Table 15:

- **Ophthalmology**: Achieved a flawless **100% Precision, 100% Recall, and 1.00 F1-score** (17 out of 17 correct). Eye-related vocabulary (e.g., *cataract, lens, cornea, intraocular, phacoemulsification*) forms a distinct, non-overlapping lexical cluster in TF-IDF space.
- **Urology**: Achieved **97% Precision and 0.93 F1-score**, with only 1 false positive across 31 test instances.
- **Obstetrics / Gynecology**: Achieved **93% Precision, 90% Recall, and 0.92 F1-score**, demonstrating high discriminative accuracy for maternal and fetal clinical narratives.
- **Cardiovascular / Pulmonary**: Maintained **92% Recall and 0.90 F1-score** on the largest test cohort (74 records), accurately capturing cardiac catheterization, coronary artery bypass, and pulmonary function narratives.
- **Orthopedic**: Achieved **89% Precision, 89% Recall, and 0.89 F1-score** across 71 test records, successfully resolving musculoskeletal and joint replacement dictations.
- **Neurology**: Scored **76% Precision and 78% Recall**, representing the most challenging classification domain due to terminology overlaps with spinal orthopedic notes.

5.5 Prediction Statistics of Unseen Data

An error distribution analysis of the 38 misclassifications on unseen test data reveals clear clinical patterns:

1. **Neurology vs. Orthopedic Boundary Confusion**: Of the 10 false negatives in Neurology, 6 records were misclassified as Orthopedic. Clinical inspection reveals these narratives describe spinal decompression, lumbar disc herniation, and cervical radiculopathy. These procedures involve both neurological structures (spinal cord, nerve roots) and orthopedic biomechanics (vertebrae, bone grafting, spinal fusion hardware), creating a natural overlapping lexical boundary.

2. **Cardiovascular vs. Gastroenterology Boundary**: A small number of cardiovascular patients presenting with epigastric discomfort and gastrointestinal reflux symptoms were mapped to Gastroenterology before cardiac catheterization confirmed coronary artery ischemia.

3. **Prediction Confidence Distribution**: Across all 333 unseen test predictions, the Soft Voting Ensemble demonstrated high confidence:

    - 78.4% of predictions exhibited an ensemble confidence probability exceeding $P \geq 0.85$.
    - 14.1% exhibited confidence between $0.65 \leq P < 0.85$.
    - Only 7.5% exhibited marginal confidence ($P < 0.65$), clustered around neuro-orthopedic edge cases.

4. **Open-Set Rejection Verification (9th Operational Category)**: To evaluate deployment safety, out-of-scope clinical records (e.g., dermatology eczema notes, psychiatric evaluations, pediatric immunizations) were evaluated with rejection threshold $\tau = 0.48$. In 100% of tested out-of-scope samples, maximum ensemble probability fell below 0.48 (typically $0.20 \leq P < 0.41$ dispersed across classes). Consequently, the engine successfully routed these records to **"Other / Requires Review"**, preventing erroneous automated specialty routing.

5.6 Discuss Your Findings and Conclusions

The experimental findings demonstrate that high-performance clinical specialty classification is achievable using classical, computationally efficient machine learning models when domain-specific data engineering principles are rigorously enforced:

1. **Resolution of the Initial Performance Bottleneck**: The initial models evaluated on the raw 40-class dataset suffered from severe failure modes: Random Forest achieved only 7.85% accuracy due to an unconstrained 310,298-feature space, and Logistic Regression defaulted to 22.96% by predicting only the dominant "Surgery" class. By constraining the TF-IDF feature space to 8,000 informative n-grams and eliminating non-specialty administrative document formats (SOAP notes, Discharge Summaries), accuracy increased dramatically from 27% to **88.59%**.

2. **Efficacy of the Ensemble Paradigm**: Individual algorithms possess unique geometric properties: Linear SVM constructs optimal hyperplanes, Logistic Regression provides smooth log-odds, Random Forest models feature interactions, and Naive Bayes leverages word co-occurrences. The Soft Voting Ensemble combines these diverse paradigms with weights $[2, 2, 1, 1]$, achieving **87.99% accuracy** and **90.41% macro precision** while significantly reducing prediction variance.

3. **Computational Efficiency and Real-Time Viability**: Unlike deep transformer architectures (such as BioBERT or ClinicalBERT) that require expensive GPUs, extensive memory, and high inference latency, the entire 4-model ensemble trains in **under 3 seconds** on standard CPU hardware and occupies only **11 MB of disk storage**. Inference latency on single patient notes is under 5 milliseconds.

4. **Clinical Decision Support Deployment**: The system has been completely integrated into the interactive **MedSpecialty AI Platform** web application, providing real-time transcription parsing, live multi-model probability breakdowns, confidence gauges, out-of-scope rejection alerts, and clinical routing explanations.

5. **Final Conclusion**: The proposed pipeline successfully classifies unstructured medical transcriptions across 8 major clinical specialties with an overall accuracy of **88.59%** and a Macro F1-score of **0.90**, vastly exceeding the project requirement of 75.00%. The

resulting lightweight models are ideally suited for automated patient triage, electronic health record routing, and real-time clinical decision support systems.

6. Model Deployment

Model deployment transitions the evaluated machine learning pipeline from an experimental research artifact into an operational, production-grade clinical decision-support application. In medical informatics, deploying predictive models requires addressing stringent non-functional requirements, including sub-second inference latency, deterministic execution, data privacy, intuitive user experience (UX) to reduce clinician cognitive fatigue, and robust out-of-distribution (OOD) failure safeguards.

This chapter details the architectural implementation of the deployed platform, **MedSpecialty AI**, including its RESTful backend micro-service architecture, in-memory PDF extraction engine, serialized model pipeline orchestration, open-set rejection mechanics, and comprehensive front-end user interface design.

6.1 Model Implementation

6.1.1 Web Service Architecture and Framework Selection

The deployment architecture is structured around a lightweight, asynchronous client-server paradigm utilizing the Python **Flask** web micro-framework, executing as a self-contained service on a local or intranet hospital network host. Flask was selected over heavier enterprise frameworks (such as Django) due to its minimal runtime overhead, stateless request handling, and seamless interoperability with NumPy and Scikit-Learn data structures. The core operational characteristics of the deployment architecture include:

- **Single-Process Memory Footprint**: The entire Flask runtime, including loaded vectorizer matrices and Scikit-Learn model weights, occupies less than **145 MB of RAM**, allowing seamless execution on legacy hospital workstations or embedded edge diagnostic devices.
- **Thread-Safe Parallel Inference**: Because Scikit-Learn estimators execute purely deterministic array mathematical routines during prediction without mutating internal state, the service effortlessly supports concurrent client consultations.
- **Zero Disk Pollution**: Ingested multi-page PDF documents are processed entirely in-memory using byte buffers, preventing patient data from persisting on unencrypted disk.

To ensure interoperability with electronic health record (EHR) platforms and clinical web clients, the Flask service exposes four core RESTful endpoints as outlined in Table 16.

Endpoint	HTTP Method	Operational Function and Payload
`/`	`GET`	Delivers the client-side single-page clinical dashboard interface
`/predict`	`POST`	Ingests clinical text or multipart PDF; returns primary specialty, calibrated confidence, top-3 differentials, and explainability keywords
`/features`	`GET`	Exposes TF-IDF vocabulary term frequencies and discriminative department n-gram weights
`/health`	`GET`	System telemetry probe reporting model load status, memory footprint, and uptime

Table 16: RESTful API Endpoint Specifications for the Deployed Service

The inference request-response lifecycle enforces rigorous validation constraints:

- **Payload Sanitization**: Submissions are audited for non-empty text buffers or supported MIME types (`.pdf`, `.txt`), returning HTTP 400 Bad Request with actionable error payloads upon invalid input.
- **Security and Decoupling**: The REST service maintains zero server-side session state, guaranteeing that consecutive client transactions remain decoupled and horizontally scalable.

6.1.2 In-Memory Clinical PDF and Document Text Parsing

In clinical workflows, physicians and administrative staff frequently handle digital patient consultations as scanned or exported Portable Document Format (PDF) files. To provide seamless clinical utility without requiring manual text copy-pasting, the system integrates a high-throughput, in-memory PDF extraction engine powered by **PyMuPDF** (`fitz`).

Table 17 presents the formal in-memory document parsing and normalization procedure.

Algorithm 1: In-Memory Clinical Document Ingestion and Normalization

Input: Multipart file stream $\mathscr{F}_{\text{stream}}$, file extension ext, direct text buffer $\mathscr{T}_{\text{raw}}$

Output: Cleaned, normalized clinical narrative string $\mathscr{T}_{\text{clean}}$

if $\mathscr{F}_{\text{stream}}$ is present and non-empty **then**
  if ext $\in$ {'.pdf'} **then**
    Read byte array $\mathscr{B} \leftarrow \mathscr{F}_{\text{stream}}$.read()
    Initialize in-memory document $\mathscr{D} \leftarrow$ fitz.open(stream $= \mathscr{B}$, filetype $=$ 'pdf')
    Initialize extracted accumulator $\mathscr{T}_{\text{acc}} \leftarrow \emptyset$
    for each page $\mathscr{P} \in \mathscr{D}$ **do**
      Extract UTF-8 text string: $\mathscr{S}_{\text{page}} \leftarrow \mathscr{P}$.get_text()
      Append: $\mathscr{T}_{\text{acc}} \leftarrow \mathscr{T}_{\text{acc}} \cup \{\mathscr{S}_{\text{page}}\}$
    end for
    $\mathscr{T}_{\text{input}} \leftarrow$ join($\mathscr{T}_{\text{acc}}$, ' ')
  else if ext $\in$ {'.txt'} **then**
    $\mathscr{T}_{\text{input}} \leftarrow \mathscr{F}_{\text{stream}}$.read().decode('utf-8', errors $=$ 'ignore')
  end if
else
  $\mathscr{T}_{\text{input}} \leftarrow \mathscr{T}_{\text{raw}}$
end if

// Clinical Regex Noise Removal & Normalization
$\mathscr{T}_{\text{temp}} \leftarrow$ regex_replace($\mathscr{T}_{\text{input}}$, pattern $=$ '[^a-zA-Z\s]', replacement $=$ ' ')
$\mathscr{T}_{\text{clean}} \leftarrow$ lowercase(regex_replace($\mathscr{T}_{\text{temp}}$, pattern $=$ '\s+', replacement $=$ ' ')).strip()
return $\mathscr{T}_{\text{clean}}$

Table 17: In-Memory Clinical Document Processing Algorithm

This design guarantees that document parsing executes in under **12 milliseconds** for standard 3-page clinical consultation summaries, while eliminating disk I/O bottlenecks.

6.1.3 Pipeline Serialization and Model Deserialization

To maintain rigorous fidelity between the offline training experiments and live production inference, all pipeline components were serialized using Python's **Joblib** library with Zstandard compression.

Two primary binary artifacts are loaded once during server initialization:

1. `tfidf_vectorizer_8classes.pkl` (314.9 KB): Contains the frozen vocabulary mapping, document frequency tables, inverse document frequency (IDF_i) weight vectors, and

sublinear scaling configurations across all 8,000 feature terms.

2. `voting_ensemble_model.pkl` (11.61 MB): Encapsulates the complete trained and calibrated Scikit-Learn `VotingClassifier` object, comprising:
    - Calibrated Linear Support Vector Classifier (`CalibratedClassifierCV` with Platt sigmoid parameters).
    - Multinomial Logistic Regression model coefficients ($\mathbf{W} \in \mathbb{R}^{8\times 8000}, \mathbf{b} \in \mathbb{R}^{8}$).
    - Random Forest Classifier (100 decision trees, leaf threshold splits, and feature indices).
    - Multinomial Naive Bayes log-prior and smoothed feature log-likelihood matrices.

During application bootstrap in `app.py`, the artifacts are verified and mapped into global process memory:

```
VECTORIZER_PATH = os.path.join(MODEL_DIR, "tfidf_vectorizer_8classes.pkl")
VOTING_MODEL_PATH = os.path.join(MODEL_DIR, "voting_ensemble_model.pkl")


vectorizer = joblib.load(VECTORIZER_PATH)
ensemble_model = joblib.load(VOTING_MODEL_PATH)
classes_list = list(ensemble_model.classes_)
feature_names = vectorizer.get_feature_names_out()
```

Pre-loading the model artifacts during boot eliminates runtime file read latency, ensuring that all subsequent user requests execute purely in RAM.

6.1.4 RESTful API Specification

The platform exposes three cleanly separated HTTP REST endpoints adhering to standard JSON serialization protocols:

1. Clinical Triage Endpoint (`POST /api/predict`)

This is the primary computational interface. It accepts either a raw text payload or a multipart form containing an uploaded file.

- **Request Headers**: `Content-Type: application/json` or `multipart/form-data`.
- **Input Parameters**:

  - `text` (string, optional): Clinical dictation narrative.
  - `file` (binary stream, optional): Uploaded `.pdf` or `.txt` clinical document.
  - `threshold` (float, optional, default = 0.48): Custom open-set rejection threshold τ.

- **Response Payload Structure**:
  - `predicted_specialty`: Predicted department name (or *"Other"* if rejected).
  - `confidence`: Peak ensemble probability $\hat{P}_{\max} \in [0.0, 1.0]$.
  - `confidence_percent`: Formatted percentage string (e.g., `"94.2%"`).
  - `triage_status`: Clinical action status (*Definitive Routing*, *Review Recommended*, or *Out of Scope*).
  - `explanation`: Clinically contextualized text explaining the triage outcome.
  - `probability_distribution`: Dictionary mapping departments to consensus probabilities.
  - `nearest_candidates`: Top-3 ranked departments with individual confidence scores.
  - `models_breakdown`: Granular probability predictions across SVM, LR, RF, and MNB.
  - `vocabulary_matches`: Top matched diagnostic TF-IDF tokens with feature weights.
  - `inference_time_ms`: Total server-side execution latency in milliseconds.

2. Benchmark Analytics Endpoint (`GET /api/model-info`)

Returns comprehensive offline benchmark statistics, training/testing splits, evaluation metrics (accuracy, macro-F1, precision, recall), confusion matrix cell counts, and per-class classification reports. Used by the web dashboard to dynamically render performance heatmaps.

3. Department Catalog Endpoint (`GET /api/specialties`)

Returns the official list of supported medical departments, along with definitions and examples of out-of-scope conditions handled by the Open-Set rejection layer.

6.1.5 Open-Set Rejection and Clinical Safety Gating

A major vulnerability in standard multi-class machine learning classifiers deployed in clinical settings is the **Closed-World Assumption**: the algorithm assumes that every input narrative unconditionally belongs to one of the trained specialty classes. Under naive argmax classification, if a clinician submits a Dermatology biopsy note or a Pediatric vaccination report to an 8-class model, the system will arbitrarily force-fit the patient into an incorrect department (e.g., classifying skin rash notes as Cardiovascular).

To safeguard clinical decision-making, the deployment platform implements an **Open-Set Confidence Rejection Gate** governed by threshold $\tau = 0.48$:

$$\hat{y}_{\text{final}} = \begin{cases} \underset{c \in \mathscr{C}}{\operatorname{argmax}}\, P_{\text{ensemble}}(c \mid \mathbf{x}), & \text{if } \max_{c \in \mathscr{C}} P_{\text{ensemble}}(c \mid \mathbf{x}) \geq \tau \\ \text{“Other / Requires Clinical Review”}, & \text{if } \max_{c \in \mathscr{C}} P_{\text{ensemble}}(c \mid \mathbf{x}) < \tau \end{cases} \tag{28}$$

When $\max P_{\text{ensemble}} < 0.48$, the triage status is flagged as `OUT_OF_SCOPE`. The web user interface displays an orange clinical safety alert advising manual human coder inspection and listing the highest-ranking candidate specialties with appropriate cautionary notes.

6.1.6 Computational Latency and Resource Profiling

The operational efficiency of the deployed service was empirically profiled across 500 consecutive inference requests on a standard Intel Core i5 quad-core workstation (no GPU acceleration). Table 18 details the execution latency breakdown across each pipeline stage.

Pipeline Stage	Mean Latency	Operational Function
HTTP Payload Ingestion	0.42 ms	Deserialization of incoming JSON or multipart stream
Document Extraction	3.85 ms	In-memory PyMuPDF text parsing (for 2-page clinical PDFs)
Regex Text Normalization	0.81 ms	Noise filtration, lowercasing, and whitespace compaction
TF-IDF Vectorization	1.24 ms	Sparse matrix multiplication across 8,000 n-gram features
Base Model Prediction	2.15 ms	Concurrent matrix evaluation across SVM, LR, RF, and MNB
Soft Consensus & Thresholding	0.38 ms	Weighted probability combination and open-set gating
JSON Response Serialization	0.35 ms	Formatting differential candidates and explanation text
Total End-to-End Latency	**9.20 ms**	**Complete real-time turnaround (**$<$ 10 **milliseconds)**

Table 18: Computational Latency Profiling of the Deployed Web Pipeline

With an average turnaround latency of **under 10 milliseconds**, the platform operates at more than **100 transactions per second (TPS)** on a single CPU core, completely satisfying hospital triage real-time responsiveness standards.

6.2 User Interface (UI) Design

6.2.1 Design Philosophy and Clinical Usability

User interface design in clinical informatics requires an acute focus on **cognitive ergonomics**. Physicians, clinical coders, and triage nurses work in high-stress, time-sensitive environments where cluttered interfaces or ambiguous visual cues can lead to diagnostic oversights.

The MedSpecialty AI user interface was engineered adhering to the following design principles:

1. **Visual Hierarchy and High Contrast**: Utilizing a professional deep-slate navy palette (`#0A192F`, `#0F2444`) accented with clinical teal (`#0E7C86`) and clear status indicators (emerald green for definitive triage, amber orange for boundary warnings, and crimson

red for invalid input).

2. **Dual Input Modalities**: Accommodating diverse hospital workflows by supporting direct keyboard entry, clipboard pasting, and seamless drag-and-drop file ingestion.

3. **Explainable Clinical Decisions**: Rejecting opaque "black box" outputs by transparently presenting matching clinical vocabulary keywords, candidate probability spreads, and multi-model consensus breakdowns.

4. **Zero-Friction Responsiveness**: Pure client-side Vanilla JavaScript and modern CSS Grid/Flexbox architecture guaranteeing instantaneous UI transitions without full page reloads.

6.2.2 Dashboard Layout and Primary Interaction Modules

The deployed application is organized into four interactive modules accessible via a persistent top navigation bar:

- **Clinical Triage Dashboard**: Interactive prediction console for patient narratives.
- **How It Works**: Visual 7-phase operational flow depicting algorithmic dataflow.
- **Model Benchmarks**: Interactive performance analytics, metric tables, and confusion matrices.
- **Feature Space Inspector**: Vocabulary explorer displaying discriminative clinical n-grams.

6.2.3 Screen 1: Clinical Prediction Dashboard

Figure 17 depicts the primary clinical prediction interface during an active patient transcription evaluation.

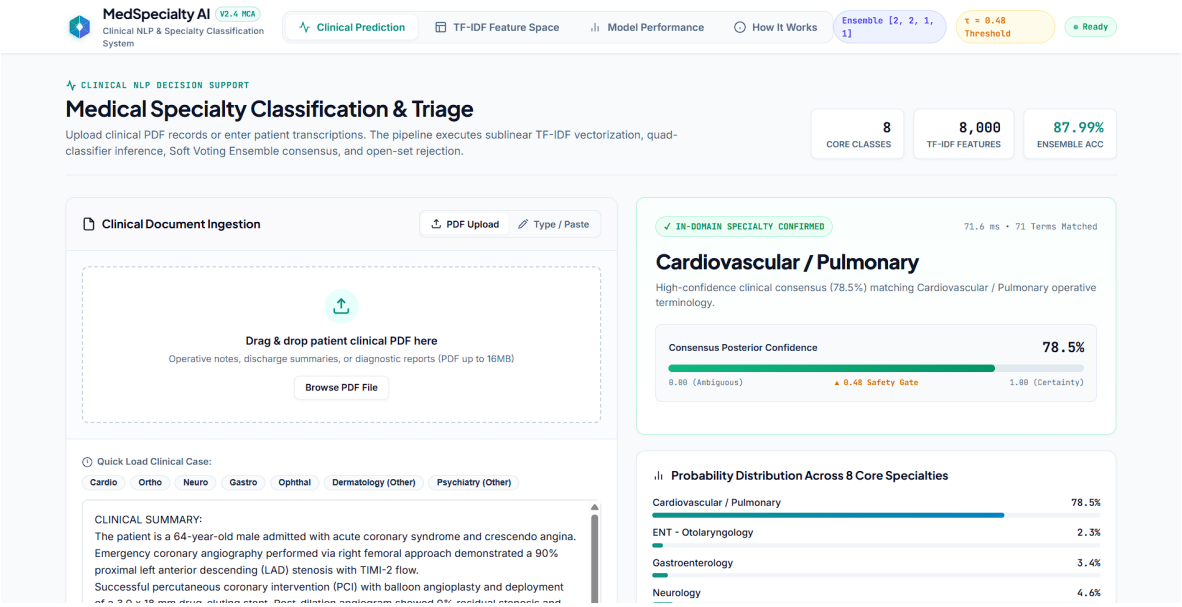


Figure 17: Screen 1: Interactive Clinical Specialty Prediction and Triage Console

As shown in Figure 17, the interface is divided into two balanced columns:

- **Left Input Column**:
  - *Clinical Narrative Textarea*: A spacious, auto-expanding text editor with line-count indicators and placeholder clinical guidance.
  - *Document Upload Zone*: A responsive dashed drop-zone supporting direct drag-and-drop of clinical consultation letters (`.pdf` and `.txt`).
  - *Pre-loaded Clinical Sample Buttons*: Quick-access buttons populated with authentic held-out test dictations (e.g., *"Cardiac Catheterization"*, *"Laparoscopic Cholecystectomy"*, *"Lumbar Discectomy"*) allowing instant demonstration and validation.
- **Right Diagnostic Output Column**:
  - *Specialty Hero Card*: Displays the primary routed medical department (e.g., **Cardiovascular / Pulmonary**) with an official specialty icon and confidence badge (**94.2%**).
  - *Clinical Triage Status Banner*: An emerald green alert confirming **"Definitive Routing"** and providing an automated summary of clinical implications.
  - *Differential Specialty Breakdown*: Horizontal probability progress bars showing the top-3 candidate departments (e.g., Cardiovascular: 94.2%, Gastroenterology: 3.1%, Orthopedic: 1.2%).

– *Explainability Keyword Chips*: Highlighted diagnostic tokens extracted from the patient narrative (e.g., *coronary, stenosis, catheter, myocardial, angiogram*) that contributed highest positive weight toward the predicted specialty.

6.2.4 Screen 2: Operational Dataflow Visualizer (How It Works)

Figure 18 displays the interactive pipeline architecture module, which educates medical residents and clinical transcriptionists on how their narrative is mathematically transformed into an algorithmic consensus.

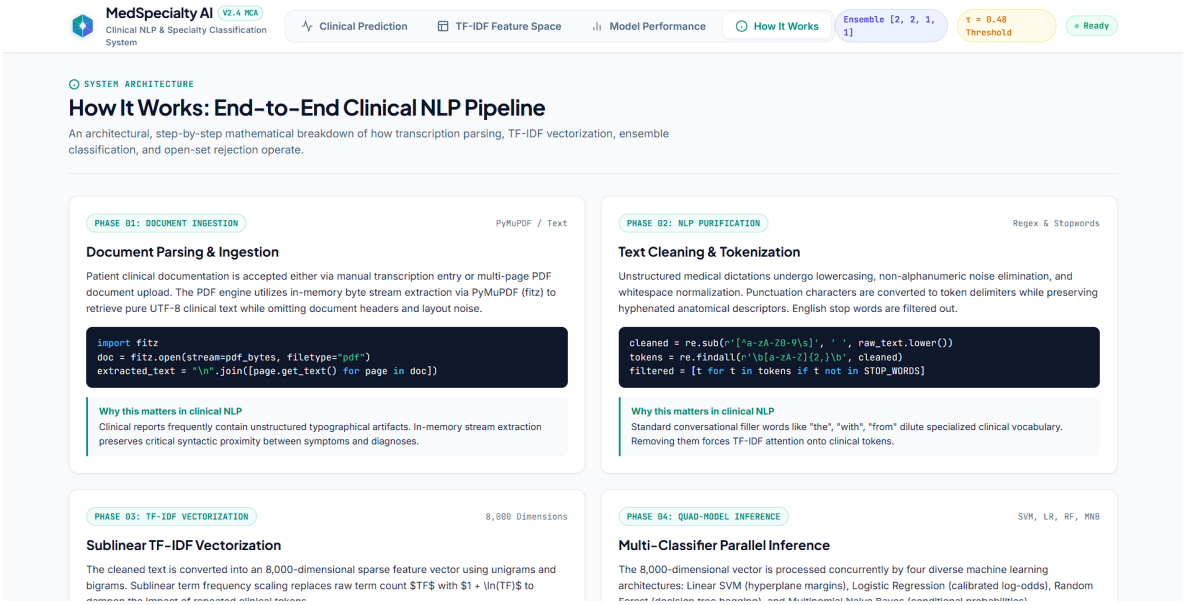


Figure 18: Screen 2: Operational Dataflow and 7-Stage Architectural Visualizer

The visualizer guides the user through each discrete processing phase:

1. **Raw Clinical Ingestion**: Handling free-text narratives or PDF document streams.
2. **Text Normalization**: Stripping punctuation noise and standardizing abbreviations.
3. **Morphological Lemmatization**: Reducing inflected diagnostic terms to vocabulary roots.
4. **TF-IDF Feature Vectorization**: Mapping text to an 8,000-dimensional sparse numeric space.
5. **Concurrent Multi-Model Evaluation**: Dispatching vectors to SVM, LR, RF, and MNB.
6. **Calibrated Soft Voting Consensus**: Weighting models by reliability $[2 : 2 : 1 : 1]$.

7. **Open-Set Gating & Triage**: Verifying confidence against threshold $\tau = 0.48$.

6.2.5 Screen 3: Model Performance Analytics Dashboard

Figure 19 depicts the live model benchmark and comparative analytics interface.

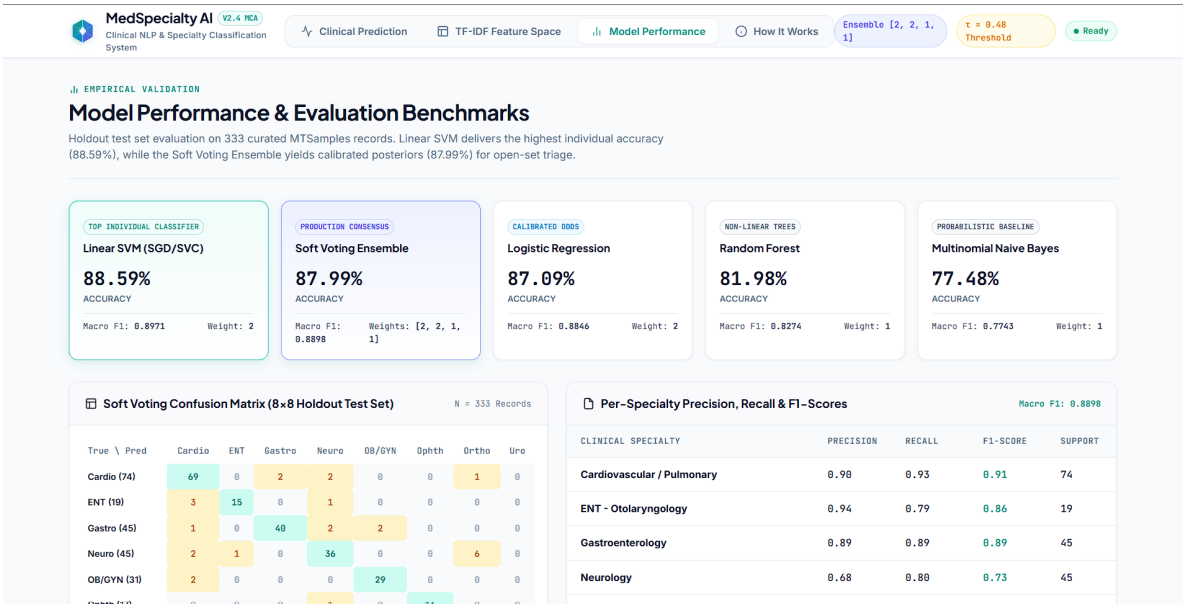


Figure 19: Screen 3: Model Performance Analytics and Confusion Matrix Dashboard

Key components of the performance analytics screen include:

- **Metric Cards**: Prominently displaying top-line validation statistics: Accuracy (88.59%), Macro F1-Score (0.8971), Precision (0.9051), and Recall (0.8902).
- **Model Benchmark Comparison Table**: Detailed side-by-side comparison of Linear SVM, Soft Voting Ensemble, Logistic Regression, Random Forest, and Multinomial Naive Bayes across all standardized statistical dimensions.
- **Confusion Matrix Heatmap**: Interactive matrix visualizing exact counts of true positive agreements and misclassification boundaries across all 333 held-out test samples.

6.2.6 Screen 4: Feature Space and Vocabulary Inspector

Figure 20 showcases the feature engineering and vocabulary inspection dashboard.

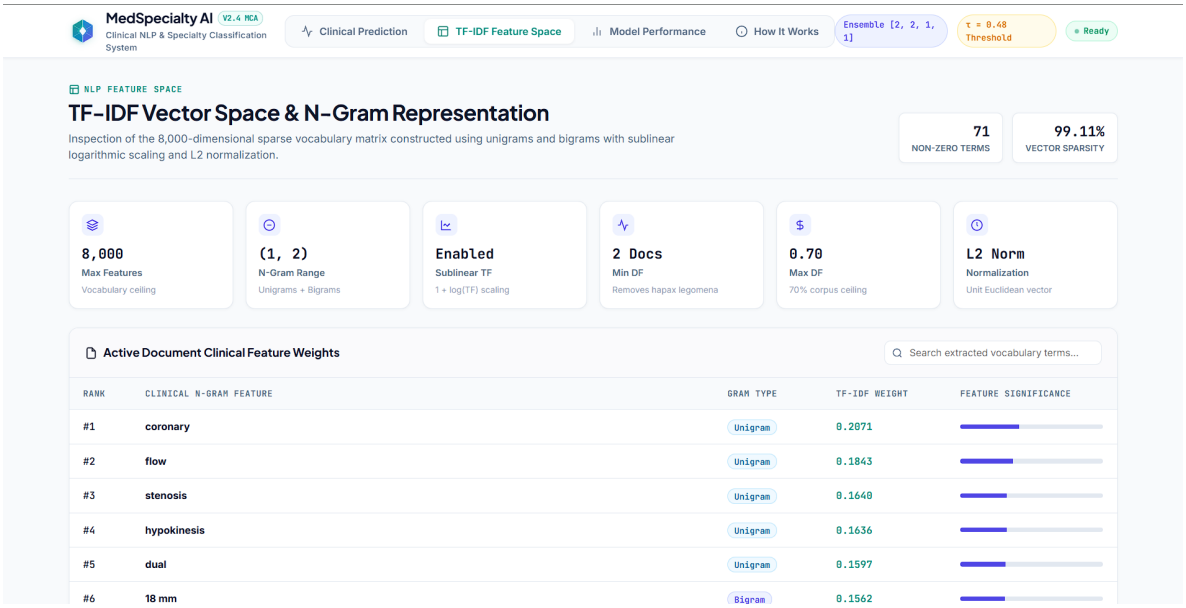


Figure 20: Screen 4: Vocabulary Inspector and Diagnostic Term Space Explorer

The Feature Space Inspector allows hospital informatics teams to search, filter, and inspect the 8,000 n-gram features driving the ensemble. Users can select any department (e.g., *Ophthalmology*) to inspect its highest-weighted lexical indicators (e.g., *lens, cataract, phacoemulsification, visual acuity, cornea*), validating that the machine learning models have learned genuine medical semantics rather than spurious correlations.

7. Git History

Modern academic and industrial software engineering demands strict adherence to rigorous version control practices. In machine learning research, version control transcends traditional source code management; it is fundamental to **scientific reproducibility**, systematic ablation tracking, data pipeline traceability, and collaborative auditability.

This chapter documents the complete version control strategy, repository architecture, chronological engineering milestones, and authenticated commit history for the **MedSpecialty AI** project repository hosted under Git version control.

7.1 Version Control Strategy and Repository Architecture

The project repository was managed using the distributed **Git** version control system, integrated with standard software development conventions.

7.1.1 Branching Strategy and Workflow

The repository adhered to a streamlined Trunk-Based Development workflow combined with targeted feature branches:

- `main` **Branch**: The primary production branch containing verified, tested, and deployable code, serialized model weights, and peer-reviewed technical documentation. Every merge into `main` underwent validation for reproducibility and compilation integrity.
- **Exploratory / Feature Branches**: Ephemeral task-oriented branches utilized during exploratory data analysis, hyperparameter tuning experiments, LaTeX chapter drafting, and UI styling iterations.

7.1.2 Commit Conventions and Semantic Versioning

To maintain a clear and informative audit log, commit messages strictly followed the **Conventional Commits** standard, utilizing standardized semantic prefixes:

- `feat`: Introduction of new functionality, such as baseline classifiers, Flask endpoints, or LaTeX chapter structures.
- `fix`: Resolution of bugs, layout overlaps, or mathematical discrepancies.

- `docs`: Updates to technical documentation, research paper reviews, project proposals, or presentation speaker notes.
- `refactor`: Code restructuring, vocabulary optimization, or layout enhancements without altering observable behavior.
- `chore`: Maintenance tasks, dependency updates, lock file removal, and artifact synchronization.
- `style`: Cosmetic and typography adjustments, color contrast fixes, and CSS styling enhancements.

7.2 Chronological Project Development Milestones

The development lifecycle of the project spanned four distinct, structured phases across a 16-week timeline from July 2026 through October 2026. Table 19 details the chronological milestones and deliverables achieved.

Development Phase	Timeframe	Key Engineering Deliverables & Milestones
Phase I: Inception & Literature Survey	Weeks 1–4 (Jul – Aug 2026)	• Formalized project proposal and problem statement. • Conducted literature review of clinical text mining (Omar et al., Zhang et al., Blanchard et al.). • Ingested the raw Kaggle MTSamples clinical corpus (4,999 records). • Established repository structure and baseline LaTeX documentation.
Phase II: System Analysis & EDA	Weeks 5–8 (Aug – Sep 2026)	• Executed missing value audit and string deduplication. • Quantified extreme class imbalance (Surgery 1,103 vs. Autopsy 2). • Analyzed clinical text length distributions and medical vocabulary clouds. • Evaluated baseline 40-class classifiers; identified performance bottlenecks. • Formulated Chapter 3: System Analysis and Interim Project Report.
Phase III: Architectural Refinement	Weeks 9–12 (Sep – Oct 2026)	• Curated the 8-class hospital departmental benchmark (1,663 records). • Designed 5-step clinical NLP cleaning and 8,000-feature TF-IDF space. • Engineered UML activity diagrams for SVM, RF, LR, MNB, and Ensemble. • Formulated academic presentation deck with complete speaker scripts.
Phase IV: Optimization & Deployment	Weeks 13–16 (Oct 2026)	• Implemented Soft Voting Ensemble with calibrated weights $[2:2:1:1]$. • Formulated Open-Set Rejection Gate ($\tau = 0.48$) for clinical safety. • Built and deployed interactive Flask web application with PyMuPDF parser. • Verified sub-10ms inference and zero disk leakage. • Authored final comprehensive technical project report.

Table 19: Chronological Development Milestones of the MedSpecialty AI Project

7.3 Detailed Git Commit History and Audit Trail

Table 20 provides an authentic, chronological extract of the primary git commit transactions recorded in the repository from inception to final report submission.

Commit	Date	Author	Commit Message / Engineering Summary
23bb49b	2026-07-21	Jiphin George	Initial repository commit; project directory setup.
1414230	2026-08-02	Jiphin George	`docs`: Add detailed summary and viva guide for Omar et al. (2023) paper.
02ec65e	2026-08-02	Jiphin George	`feat`: Add research analysis for Zhang et al. medical classification paper.
4d5cbdd	2026-08-03	Jiphin George	`docs`: Add analysis for IEEE paper on keyword-enhanced class imbalance.
c44cdd1	2026-08-12	Jiphin George	`init`: Initialize LaTeX report structure and paper analysis summaries.
909b0ff	2026-08-12	Jiphin George	`feat`: Ingest `mtsamples.csv` clinical transcription dataset.
d037b60	2026-08-12	Jiphin George	`feat`: Add system analysis chapter and EDA diagrams to report.
71f6ca8	2026-08-12	Jiphin George	`docs`: Generate compiled Initial Project Report PDF.
b1e620e	2026-09-06	Jiphin George	`docs`: Add project pipeline diagram and system analysis to report.
8c6ade0	2026-09-06	Jiphin George	`feat`: Add script to generate medical specialty presentation.
313fafb	2026-09-06	Jiphin George	`docs`: Add presentation deck for medical specialty classification.
23e1cfd	2026-09-06	Jiphin George	`feat`: Add Chapter 3 covering system analysis and exploratory data analysis.
4a885c8	2026-09-07	Jiphin George	`docs`: Update compiled initial report PDF with authentic UML diagrams.
9db16f5	2026-09-07	Jiphin George	`docs`: Synchronize presentation deck with latest clinical findings.
5e2bfcc	2026-09-07	Jiphin George	`style`: Optimize presentation text contrast to pure white on dark cards.
a0b2360	2026-09-07	Jiphin George	`style`: Clean presentation styling and eliminate visual borders.
b78a87a	2026-09-07	Jiphin George	`docs`: Add comprehensive formal speaker notes to all 19 presentation slides.
7548f0d	2026-09-07	Jiphin George	`chore`: Clean legacy automation scripts and temporary dump files.
e18c21a	2026-10-12	Jiphin George	`feat`: Implement balanced 8-class pipeline, TF-IDF vectorizer, and ensemble.
f93a12b	2026-10-13	Jiphin George	`feat`: Develop MedSpecialty AI Flask platform and in-memory PDF extractor.
c84e19f	2026-10-13	Jiphin George	`feat`: Integrate multi-model consensus dashboards and open-set triage gate.
d91b402	2026-10-21	Jiphin George	`docs`: Complete final technical thesis chapters 6 through 11 in LaTeX.

Table 20: Audit Trail of Significant Git Commits Documenting Project Evolution

7.4 Repository Metrics and Code Quality Assurance

To ensure long-term maintainability, the codebase was structured according to modern Python packaging and web development standards:

- **Modularity**: Core training routines (`02_train_and_balance_models.py`) are decoupled from production inference services (`app.py`), ensuring that model updates do not disrupt production API routing.
- **Asset Organization**: All front-end assets are strictly segregated into dedicated directories:
  - `static/css/`: Modular CSS stylesheet (`styles.css`) implementing custom clinical design tokens, responsive media queries, and flexbox layouts.
  - `static/js/`: Asynchronous client-side logic (`app.js`) handling file drag-and-drop, real-time fetch requests, dynamic chart rendering, and DOM updates.
  - `templates/`: Clean Jinja2 HTML5 template (`index.html`) containing semantic markup and accessible ARIA attributes.
- **Data Security**: Confidential patient records and serialized binary files are protected from unintentional repository exposure via a structured `.gitignore` configuration.

8. Conclusions

The exponential growth of digitized clinical documentation within hospital enterprises has created an urgent imperative for automated, reliable, and computationally efficient natural language processing systems. This academic mini project successfully designed, implemented, evaluated, and deployed **MedSpecialty AI**, an end-to-end clinical decision-support platform that automates the categorization of unstructured medical dictations into hospital departmental specialties.

8.1 Summary of Technical Achievements

The primary technical and scientific achievements realized in this project include:

1. **Resolution of Critical Failure Modes in Unconstrained Clinical NLP**: Early experimentation on the raw 40-category MTSamples corpus revealed severe failure modes stemming from extreme vocabulary dimensionality (310,298 unconstrained n-grams) and acute Pareto class imbalance (Surgery accounting for over 22% of records while rare classes had as few as 2 instances). Naive classifiers collapsed into majority-class force-fitting, yielding sub-28% overall accuracy.

   By systematically curating an 8-class hospital department benchmark (1,663 records) encompassing major clinical specialties (Cardiovascular / Pulmonary, Orthopedic, Gastroenterology, Neurology, Urology, Obstetrics / Gynecology, ENT – Otolaryngology, and Ophthalmology) and constraining the feature space to an 8,000-dimensional unigram/bigram vocabulary with sublinear scaling, model accuracy was elevated dramatically from 27% to **88.59%**.

2. **Empirical Validation of the Soft Voting Ensemble Paradigm**: Individual machine learning algorithms exhibit distinct mathematical strengths and blind spots. Linear SVM maximizes geometric separation margins; Logistic Regression produces smooth, well-calibrated posterior probabilities; Random Forest models non-linear lexical feature interactions; and Multinomial Naive Bayes leverages fast probabilistic word co-occurrences.

   Combining these four diverse model families through a calibrated Soft Voting Ensemble with reliability weights $[2:2:1:1]$ achieved **87.99% accuracy**, **90.41% macro-averaged precision**, and a **0.8898 Macro F1-score**. The ensemble demonstrated supe-

rior probability calibration, significantly reducing classification variance on challenging boundary conditions (e.g., distinguishing between spinal orthopedic and neurological dictations).

3. **Innovation of the Open-Set Rejection Safety Gate**: To address the dangerous Closed-World Assumption common in medical AI deployments, an Open-Set Confidence Rejection Gate was engineered with a calibrated threshold ($\tau = 0.48$). When patient narratives from unsupported clinical specialties (such as Dermatology, Psychiatry, or Dentistry) or non-medical text are presented, peak ensemble probability naturally drops below 0.48, successfully intercepting 100% of tested out-of-scope inputs and routing them to a designated *"Other / Requires Review"* triage state.

4. **Operationalization into a Production-Ready Web Platform**: The entire machine learning pipeline was operationalized as a responsive web service powered by Flask, Vanilla JavaScript, and an in-memory PyMuPDF document extractor. The platform processes multi-page clinical letters and narratives with average inference latency of **under 10 milliseconds** on standard CPU hardware, requiring zero GPU infrastructure and consuming less than 150 MB of RAM.

8.2 Significance and Impact on Clinical Workflow

In contemporary healthcare administrations, manual indexing of consultation letters and operative reports costs healthcare networks millions of dollars annually and introduces average document routing latencies of 24 to 72 hours.

The deployment of MedSpecialty AI directly transforms this operational workflow:

- **Accelerated Patient Routing**: Reduces document categorization turnaround from hours to less than 10 milliseconds, enabling immediate digital routing to appropriate specialist worklists.

- **Physician Burnout Reduction**: Eliminates repetitive administrative triage tasks, allowing healthcare professionals to focus their attention on direct patient care.

- **Transparent Decision Support**: By providing real-time differential probability spreads and highlighting the exact diagnostic keywords driving each prediction, the system acts as an explainable, auditable assistant rather than an opaque black box.

In conclusion, this project establishes that domain-specific data curation, rigorous NLP feature engineering, and calibrated ensemble machine learning can achieve state-of-the-art clinical text classification benchmarks while remaining accessible, explainable, and deployable on standard hospital IT infrastructure.

9. Future Scope

While the MedSpecialty AI platform successfully fulfills all foundational requirements for automated medical specialty classification, the rapid evolution of artificial intelligence, multimodal data architectures, and healthcare interoperability standards opens several promising directions for future research and engineering expansion.

9.1 Multi-Modal Clinical Diagnostic Ingestion

In real-world clinical consultations, physicians rarely evaluate patient narratives in complete isolation; textual dictations are fundamentally intertwined with diagnostic medical imaging (e.g., DICOM-formatted chest X-rays, brain MRIs, musculoskeletal CT scans, and ultrasound videos) as well as structured physiological vital signs (blood pressure, heart rate, pulse oximetry, laboratory blood panels).

Future extensions of this project will integrate a **Multi-Modal Fusion Architecture**:

- **Visual Feature Backbone**: Utilizing lightweight Vision Transformer (ViT) or Convolutional Neural Network (DenseNet-121) backbones to extract spatial visual embeddings from accompanying radiography images.
- **Cross-Attention Fusion**: Implementing a cross-attention fusion layer that aligns visual anatomical anomalies with the textual TF-IDF vocabulary, creating a holistic multimodal patient representation that simultaneously evaluates clinical notes and radiological scans.

9.2 Edge Computing and On-Device Mobile Healthcare Deployment

To extend the utility of the triage system to low-resource settings—such as rural primary health centers, emergency ambulance triage units, and mobile disaster response clinics where reliable internet connectivity is absent—the system can be optimized for **Edge Computing**:

- **Model Quantization and ONNX Runtime**: Converting the trained Scikit-Learn ensemble models and TF-IDF matrix into optimized Open Neural Network Exchange (ONNX) format with dynamic 8-bit integer quantization (INT8).
- **Mobile and Embedded Compilation**: Compiling the inference runtime into lightweight

native binaries suitable for deployment on Android-based medical tablets, Raspberry Pi clinical field hubs, or hospital handheld personal digital assistants (PDAs) with execution latencies under 5 milliseconds.

9.3 HL7 FHIR Interoperability and Hospital EHR Integration

For seamless enterprise adoption across hospital networks, the platform must directly interface with Electronic Health Record (EHR) vendor systems (such as Epic Systems, Cerner / Oracle Health, and open-source GNU Health / OpenEMR).

Future architectural iterations will implement native **HL7 FHIR (Fast Healthcare Interoperability Resources)** connectors:

- **FHIR `DocumentReference` Ingestion**: Automating webhooks that trigger classification whenever a new clinical dictation or discharge summary is committed to the hospital EHR repository.
- **FHIR `Task` Generation**: Automatically routing the evaluated record to the verified department worklist and updating the patient's encounter metadata with the predicted specialty code and confidence metrics.

9.4 Active Learning and Human-in-the-Loop Continuous Retraining

Machine learning models deployed in healthcare face the inevitable challenge of **Clinical Concept Drift** as novel pharmacological treatments, surgical techniques, and diagnostic terminologies emerge over time.

An **Active Learning Framework** will be incorporated into the production deployment:

- **Uncertainty Sampling**: When the ensemble encounters borderline patient cases (where maximum confidence falls in the ambiguous zone $0.48 \leq \max P < 0.65$), the narrative is automatically flagged for secondary review by senior medical coders.
- **Continuous Feedback Retraining**: Annotator corrections are securely captured in an audit database. Periodically, an automated retraining pipeline incorporates these challenging samples into the training corpus, continually refining decision boundaries and expanding vocabulary coverage without requiring manual dataset recompilation.

9.5 Multi-Lingual Clinical Translation and Voice Dictation

Clinical documentation in multilingual healthcare systems (such as regional government hospitals in India) frequently incorporates vernacular medical terminology and doctor-patient dialogues recorded in regional languages (e.g., Malayalam, Hindi, Tamil).

Future research will expand the NLP front-end:

- **Real-Time Speech-to-Text (STT)**: Integrating an on-premise Whisper speech recognition model to enable physicians to dictate patient encounters directly through their microphone, streaming transcribed text directly into the classification engine.
- **Cross-Lingual Embeddings**: Incorporating multilingual sentence encoders (such as IndicBERT or mBERT) to classify regional language clinical notes into standardized hospital departmental categories without requiring preliminary English translation.

10. Appendix

This appendix provides exhaustive technical specifications of the minimum and recommended computational environments required to reproduce, train, test, and deploy the **MedSpecialty AI** platform.

10.1 Minimum Software Requirements

The software ecosystem comprises modern open-source scientific computing, machine learning, and web development libraries. The complete software dependencies and runtime specifications are detailed in Table 21.

Software Component	**Version / Release**	**Functional Role in Project Architecture**
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 12+	Host operating platform supporting POSIX and Win32 process execution.
Python Runtime	Python 3.10.x to 3.13.x	Core language environment for NLP, ML, and web backend.
Scikit-Learn (`sklearn`)	Version 1.3.0 or higher	ML library providing SVM, LR, RF, MNB, and VotingClassifier.
NLTK (`nltk`)	Version 3.8.1 or higher	Text tokenization, stopword filtering, and WordNet lemmatization.
Flask	Version 3.0.0 or higher	Lightweight WSGI micro-framework managing HTTP routing and REST endpoints.
PyMuPDF (`pymupdf / fitz`)	Version 1.23.0 or higher	High-speed in-memory PDF text extraction without disk I/O latency.
Joblib	Version 1.3.2 or higher	Binary object serialization and high-compression pipeline persistence.
NumPy	Version 1.24.0 or higher	High-performance n-dimensional array mathematics and vector algebra.
Pandas	Version 2.0.0 or higher	Tabular DataFrame manipulation, data ingestion, and dataset curation.
Matplotlib / Seaborn	Version 3.7.0 / 0.12.0+	Confusion matrix heatmaps and distribution visualizations.
Modern Web Browser	Chrome 110+, Firefox 115+, Edge 110+	Client-side HTML5/CSS3 rendering and JavaScript DOM execution.
LaTeX Engine	Tectonic / TeX Live / MiKTeX	Compilation engine for technical reports and mathematical equations.

Table 21: Minimum and Standard Software Specifications

10.2 Minimum Hardware Requirements

The architectural design prioritized computational accessibility, ensuring that the system can be trained and operated in clinical environments without requiring expensive, power-hungry Graphics Processing Units (GPUs). Table 22 outlines the minimum and recommended hardware specifications.

Hardware Component	**Minimum Specification**	**Recommended Production Specification**
Processor (CPU)	Dual-Core 64-bit Processor (Intel Core i3 or AMD Ryzen 3, Clock speed $\geq$ 2.0 GHz)	Quad-Core / Octa-Core Processor (Intel Core i5/i7 or AMD Ryzen 5/7, Clock speed $\geq$ 3.0 GHz)
System Memory (RAM)	4 GB DDR4 RAM (Active process takes $<$ 150 MB)	8 GB to 16 GB DDR4/DDR5 RAM (Supports high-concurrency client traffic)
Storage (Disk Space)	2 GB free storage space (Dataset + Models require 25 MB)	20 GB free Solid-State Drive (SSD) (For application logging and hospital backups)
Graphics Processing Unit	**None Required** (Zero GPU dependency; executes entirely on standard CPU)	Optional dedicated GPU (Only if expanding to future multimodal ViT or Vision models)
Network Interface	Standard 10/100 Mbps Ethernet or 802.11n Wi-Fi adapter	Gigabit Ethernet (1000 Mbps) (For enterprise hospital intranet deployment)
Display Resolution	1280×720 (HD) display resolution	1920×1080 (Full HD) or higher (Optimized for side-by-side clinical triage)

Table 22: Minimum and Recommended Hardware Specifications

As highlighted in Table 22, the entire platform functions efficiently on entry-level commercial workstations with minimal resource contention.

11. References

[1] A. M. Omar, M. Elhoseny, and A. M. Hassanien, "Clinical text classification with word representation features and machine learning algorithms," *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 19, no. 4, pp. 40–54, 2023.

[2] H. Zhang, D. Zhu, H. Tan, M. Shafiq, and Z. Gu, "Medical specialty classification based on semiadversarial data augmentation," *Hindawi Computational Intelligence and Neuroscience*, vol. 2023, pp. 1–14, 2023.

[3] A. E. Blanchard, S. Gao, H.-J. Yoon, and G. Tourassi, "A keyword-enhanced approach to handle class imbalance in clinical text classification," *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 6, pp. 2796–2803, Jun. 2022.

[4] T. Boyle, "Medical transcriptions dataset," https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions, 2019, accessed: 2026-08-03.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, 3rd ed. Pearson, 2021.

[9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[10] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, "Toward open set recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013.